Solving Billion-Scale Guaranteed Delivery Advertising Allocation

Anonymous Author(s)

Abstract

Guaranteed Delivery (GD) advertising is a fundamental component of online marketing, where inventory allocation plays a critical role in determining the revenue and profit of advertising platforms. Nowadays, with the rapid growth of commercial demands and the surge of traffic during peak events such as large-scale promotions, GD allocation in real-world platforms has reached an unprecedented scale, involving nearly one hundred million users—each representing a supply of impressions—to be allocated across thousands of advertising contracts, resulting in a billion-level allocation scale. However, existing methods struggle to deliver high-quality allocations at such billion scale in practice. To address these challenges, we propose PILLS, an end-to-end framework centered on an offline decompose–solve–recover pipeline with online incremental updates. In its offline pipeline, PILLS decomposes the original problem into subproblems, solves them in parallel using lightweight local search, and merges their solutions through a cross-subproblem recovery procedure. It further incorporates an incremental algorithm to support online real-time updates. Theoretical results are also established for the proposed algorithms, including probabilistic bounds, approximation guarantees, and local-optimality properties. PILLS has been deployed in the Alibaba GD advertising system for one year. Evaluations on billion-scale real data and the performance of the deployed system demonstrate that PILLS consistently outperforms existing algorithms, delivering significant improvements in delivery rate, revenue, and CPM. To the best of our knowledge, this is the first algorithm capable of handling GD advertising allocation at the billion scale under practical business time limits.

CCS Concepts

• **Information systems** → **Computational advertising**.

Keywords

Guaranteed Delivery Advertising, Decomposition-based Parallel Algorithm, Local Search Algorithm

ACM Reference Format:

Anonymous Author(s). 2018. Solving Billion-Scale Guaranteed Delivery Advertising Allocation. In . ACM, New York, NY, USA, 18 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Guaranteed delivery (GD) advertising constitutes a vital component in the online advertising ecosystem, playing an essential role in brand promotion and digital commerce. A central task in GD advertising is the allocation of impressions to advertising contracts. As illustrated in Figure 1, an advertiser submits an order through the publisher's trading platform in advance of the advertising campaign and expects to receive an allocation result in real time. Therefore, the quality of allocation is critical to fulfilling advertiser demand. Indeed, even marginal improvements in delivery rate can increase publishers' revenue by tens of millions of dollars and improve advertisers' return on investment.

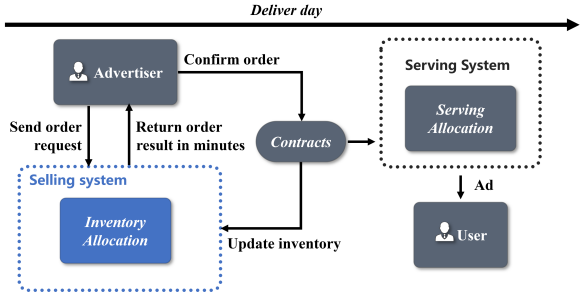


Figure 1: The system architecture for GD advertising.

Over the years, a variety of methods have been developed for GD allocation, with different trade-offs between computational efficiency and solution quality. The first category relies on mathematical programming solvers to directly solve the allocation model, which can yield exact solutions but typically suffers from long runtimes at scale. Another direction is to employ greedy heuristics such as HWM [5], which provide fast allocations, but the quality of the obtained solutions is often limited. A third class builds on primal–dual algorithms derived from KKT conditions, which iteratively update primal and dual variables based on these optimality conditions [7, 9, 31]. However, dual variable updates and gradient computations incur substantial computational overhead at ultra-large scales, limiting the scalability of such methods.

Under strict response-time requirements in GD systems, all of the above methods face limitations in the scale they can handle. To the best of our knowledge, the largest scale handled by prior studies under industrial runtime limits is a few hundred thousand supply nodes, achieved using the primal–dual method [31]. However, with the growing commercial demand and traffic surges during peak events such as large-scale promotions, real-world GD problems often involve nearly one hundred million users as supply nodes and thousands of advertising contracts, resulting in a billion-level allocation scale. At this scale, existing methods struggle in practice.

More specifically, the challenges of billion-scale GD allocation arise in both offline and online allocation. Offline allocation must compute an allocation for signed contracts at billion scale, posing the central scalability challenge, whereas online allocation must determine allocations for newly arriving contracts in real time. These challenges motivate the exploration of new algorithms and techniques for industrial-scale GD allocation.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

In this paper, we propose **PILLS**, a recoverable decomposition-based framework built on Parallel Incremental Lightweight Local Search, for high-quality, billion-scale GD advertising inventory allocation. Our main contributions are summarized as follows:

- We design an end-to-end allocation framework for billion-scale GD advertising systems. It centers on an offline decompose–solve–recover pipeline and supports online incremental updates, connecting billion-scale offline optimization with real-time allocation for newly arriving contracts.
- We develop lightweight algorithms to implement this architecture. Within the decompose–solve–recover paradigm, a two-level decomposition partitions the original problem, tailored local search algorithms efficiently solve the resulting subproblems in parallel, and a cross-subproblem recovery procedure merges their solutions to recover the loss caused by decomposition. We further design an incremental local search for real-time allocation of new contracts. Finally, we provide theoretical analyses of the proposed algorithms, including probabilistic bounds, approximation guarantees, and local-optimality properties.
- PILLS has been deployed in the Alibaba GD advertising system for one year. Evaluations on billion-scale real-world data and deployment performance demonstrate that PILLS significantly improves allocation quality, delivery rate, revenue and CPM.

1.1 Related Work

Guaranteed Delivery (GD) Advertising is widely regarded as a challenging problem in the advertising industry and has motivated substantial research. Typically, GD advertising involves two main processes: selling, in which inventory is allocated among advertisers, and serving, in which the resulting allocation plan is executed in real time to deliver impressions to users. In recent years, substantial research on GD advertising has focused on serving, including algorithms based on stochastic arrival models [2, 11, 23, 25] and impression-based algorithms [1, 3, 6, 8, 10, 15, 24], some of which exploit duality, such as SHALE [4, 5], to accelerate allocation.

In contrast, relatively little work has studied the allocation problem in selling. If problem scale and time limits are not considered, the most general way to solve GD selling allocation is to rely on off-the-shelf mathematical programming solvers such as Gurobi [14]. However, since these solvers could not quickly produce high-quality solutions, greedy heuristic methods such as HWM [5] long served as the main approach to selling, until the growing business scale exposed their inability to deliver satisfactory allocations. Accordingly, primal–dual algorithms derived from KKT conditions were proposed, which iteratively update primal and dual variables based on the optimality conditions and provide better solutions than heuristics [7, 9]. More recently, the two-stage primal–dual framework Rap was developed to reduce the number of iterations required for real-time allocation and thus speed up computation [31].

Local Search Algorithms are widely used to obtain high-quality solutions to large-scale NP-hard problems by iteratively improving candidate solutions [12, 13, 17, 22, 27–30]. They have also been applied to GD advertising in complex scenarios, such as those involving nonlinear constraints and bi-objective optimization [16, 18]. In addition, parallel local search has been explored, with existing work focusing on running multiple search strategies in parallel on the same problem [20]. Although such parallelism improves search diversity, it does not reduce the scale of the underlying problem, thereby limiting scalability for extremely large problems. To the best of our knowledge, there is no decomposition-based parallel local search algorithm for extremely large-scale GD advertising.

2 Problem Model

The GD advertising allocation problem is to compute the maximum possible impression for the new contract while ensuring the impression amounts satisfy the demand of advertisers.

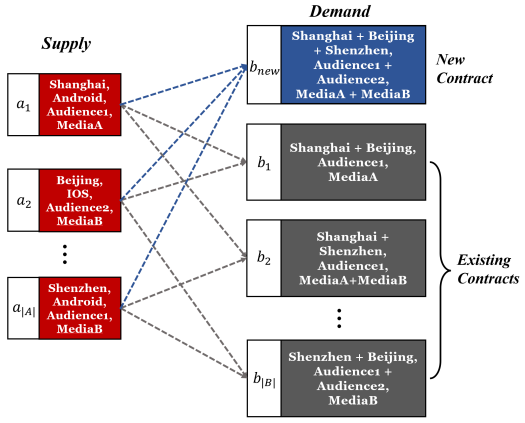


Figure 2: Bipartite graph model of GD advertising allocation

The problem can be described using the bipartite graph as in Figure 2. On the left side, the supply nodes set A represent users and their corresponding impressions, each with a capacity indicating the number of impressions available. On the right side, the demand node set B corresponds to contracts signed by advertisers, each with a specified demand for impressions. Adjacency matrix $[M_{ab}]_{a\in A,b\in B}$, where $M_{ab} = 1$ if supply node a is eligible to be allocated to demand node b. Decision variables x_{ab} represent the amount of supply allocated from a to demand b.

We reformulate the GD advertising inventory allocation into a two-stage optimization framework, consisting of an offline stage and an online stage. In the offline stage, relatively ample time is available to compute a high-quality allocation for existing contracts, with the goal of satisfying their demands as much as possible. In the online stage, new contracts arrive sequentially, and each is allocated as much supply as possible in real time.

2.1 Offline Stage

In the offline stage, the set of pre-existing contracts is denoted as B^{off}. Each supply node $a \in A$ is assigned a heat $w_a \in (0, 1)$, with higher values indicating that its impressions are valuable to a broader range of advertisers and should therefore be preserved whenever possible. For each contract $b \in B^{\text{off}}$, an under-delivery variable $u_b \geq 0$ is introduced to represent the gap between its demand and the allocated impressions. The objective f_{off} imposes a large penalty coefficient $p \gg 1$ on under-delivery while encouraging the preservation of capacity on high-heat supply nodes.

Formally, the offline allocation problem is defined as:

$$\max \quad \sum_{a\in A} w_a\Big(\text{capacity}(a) - \sum_{b\in B^{\text{off}}} M_{ab}x_{ab}\Big)^2 - p\sum_{b\in B^{\text{off}}} u_b \tag{1}$$

$$\text{s.t.} \quad \sum_{b\in B^{\text{off}}} M_{ab}x_{ab} \le \text{capacity}(a), \quad \forall a \in A, \tag{2}$$

$$\sum_{a\in A} M_{ab}x_{ab} + u_b \ge \text{demand}(b), \quad \forall b \in B^{\text{off}}, \tag{3}$$

$$x_{ab} \in \mathbb{N}, \quad \forall a \in A,\ b \in B^{\text{off}}, \tag{4}$$

$$u_b \ge 0, \quad \forall b \in B^{\text{off}}. \tag{5}$$

2.2 Online Stage

In the online stage, newly arriving contracts, denoted by B^{on}, arrive sequentially and are allocated in real time. Let $B^{\text{on}}_{\text{prev}} \subset B^{\text{on}}$ denote the previously arrived online contracts, and let b_{new} denote the newly arriving contract. For each b_{new}, the objective maximizes its allocation from the available supply while retaining the penalty for existing contracts. The resulting allocation amount becomes the demand of b_{new}, after which it is treated as an existing online contract. Formally, the online allocation problem is defined as:

$$\max \quad \sum_{a\in A} M_{ab_{\text{new}}}x_{ab_{\text{new}}} - p\sum_{b\in B^{\text{off}}\cup B^{\text{on}}_{\text{prev}}} u_b \tag{6}$$

$$\text{s.t.} \quad \sum_{b\in B^{\text{off}}\cup B^{\text{on}}_{\text{prev}}\cup\{b_{\text{new}}\}} M_{ab}x_{ab} \le \text{capacity}(a), \quad \forall a \in A, \tag{7}$$

$$\sum_{a\in A} M_{ab}x_{ab} + u_b \ge \text{demand}(b), \quad \forall b \in B^{\text{off}} \cup B^{\text{on}}_{\text{prev}}, \tag{8}$$

$$x_{ab} \in \mathbb{N}, \quad \forall a \in A,\ b \in B^{\text{off}} \cup B^{\text{on}}_{\text{prev}} \cup \{b_{\text{new}}\}, \tag{9}$$

$$u_b \ge 0, \quad \forall b \in B^{\text{off}} \cup B^{\text{on}}_{\text{prev}}. \tag{10}$$

The overall objective f_{on} for the online stage, evaluated at the end of the online stage, maximizes the total allocation maintained for all online contracts while penalizing under-delivery:

$$\max \quad \sum_{b\in B^{\text{on}}}\sum_{a\in A} M_{ab}x_{ab} - p\sum_{b\in B^{\text{off}}\cup B^{\text{on}}} u_b. \tag{11}$$

3 Offline Decompose–Solve–Recover and Online Updates

In this section, we introduce our algorithm PILLS, which efficiently addresses the ultra-large-scale allocation problem in the offline stage and handles real-time allocation for newly arriving contracts in the online stage, as illustrated in Figure 3.

In the offline stage, the original bipartite graph is extremely large and impractical to solve directly within limited time. This motivates us to decompose the graph into smaller subproblems that can be solved in parallel. The key idea is that each decomposed subproblem does not need to preserve complete global information, as long as cross-subproblem interactions can be recovered efficiently through a subsequent merge. Based on this insight, we develop a decompose–solve–recover framework for offline allocation.

In the online stage, contracts arrive sequentially and require real-time allocation. Re-solving the full problem for each arrival is impractical. We therefore use the offline solution as the baseline and update it incrementally for each new contract.

3.1 Two-Level Graph Decomposition

We decompose the original bipartite graph using a two-level decomposition. At the first level, the original graph is partitioned into structurally independent subgraphs. At the second level, each subgraph obtained at the first level is further refined by partitioning its supply nodes, resulting in smaller-scale subproblems, which are then solved in parallel.

Let $G = (A, B, E)$ denote the bipartite allocation graph, where A and B are the sets of supply and demand nodes, respectively, and $E \subseteq A \times B$ is the set of allocation edges.

Level 1: Independent subgraph. We first decompose G into a collection of disjoint subgraphs $G^{(1)}, G^{(2)}, \ldots, G^{(k)}$, such that

$$A = \bigcup_{s=1}^{k} A^{(s)}, \qquad B = \bigcup_{s=1}^{k} B^{(s)}, \qquad E = \bigcup_{s=1}^{k} E^{(s)},$$

where k is determined by the connected components of G. For any $s, s' \in \{1, \ldots, k\}$ with $s \ne s'$,

$$E^{(s)} \cap E^{(s')} = \emptyset, \qquad \forall\, a \in A^{(s)},\ b \in B^{(s')} : (a, b) \notin E.$$

As a result, no edges exist between any two subgraphs, and each $G^{(s)}$ can be solved independently without loss of optimality.

Level 2: Supply-side partitioning. In real-world GD allocation, the supply (user) side is orders of magnitude larger than the demand side. This scale asymmetry motivates us to further partition the supply set within each first-level subgraph, yielding smaller subproblems.

For a given first-level subgraph $G^{(s)}$, we uniformly shuffle its supply set $A^{(s)}$ and divide it into m disjoint subsets $\{A^{(s,i)}\}_{i=1}^{m}$. For notational simplicity, we assume that m divides $|A^{(s)}|$.[1] The resulting partition satisfies

$$|A^{(s,i)}| = \frac{|A^{(s)}|}{m}, \qquad A^{(s,i)} \cap A^{(s,j)} = \emptyset \quad \forall\, i \ne j.$$

This induces m subproblems $G^{(s,1)}, G^{(s,2)}, \ldots, G^{(s,m)}$. The number of subproblems is determined by the available parallelism: $m = \max\left\{1, \left\lfloor \frac{T}{k} \right\rfloor\right\}$, where T is the total number of available threads and k is the number of first-level subgraphs.

For each subproblem $G^{(s,i)}$, the demand set is kept unchanged and identical to that of the corresponding first-level subgraph, i.e., $B^{(s,i)} = B^{(s)}$. Its edge set is $E^{(s,i)} = \{(a, b) \in E^{(s)} : a \in A^{(s,i)}\}$, and hence $G^{(s,i)} = \left(A^{(s,i)}, B^{(s,i)}, E^{(s,i)}\right)$.

While the demand nodes are common to all subproblems, the demand requirement of each demand node is evenly distributed among the m subproblems. Specifically, for each $b \in B^{(s)}$, the demand assigned to subproblem $G^{(s,i)}$ is

$$\text{demand}^{(s,i)}(b) = \frac{\text{demand}(b)}{m}, \quad \sum_{i=1}^{m} \text{demand}^{(s,i)}(b) = \text{demand}(b).$$

We next establish theoretical guarantees for supply-side partitioning with respect to slack and expected relative underdelivery.

[1] Otherwise, the shuffled supply sequence is divided into blocks of size $\lfloor |A^{(s)}|/m \rfloor$ or $\lceil |A^{(s)}|/m \rceil$, so that their sizes differ by at most one. The subsequent theoretical analysis is also conducted under the assumption that m divides $|A^{(s)}|$.

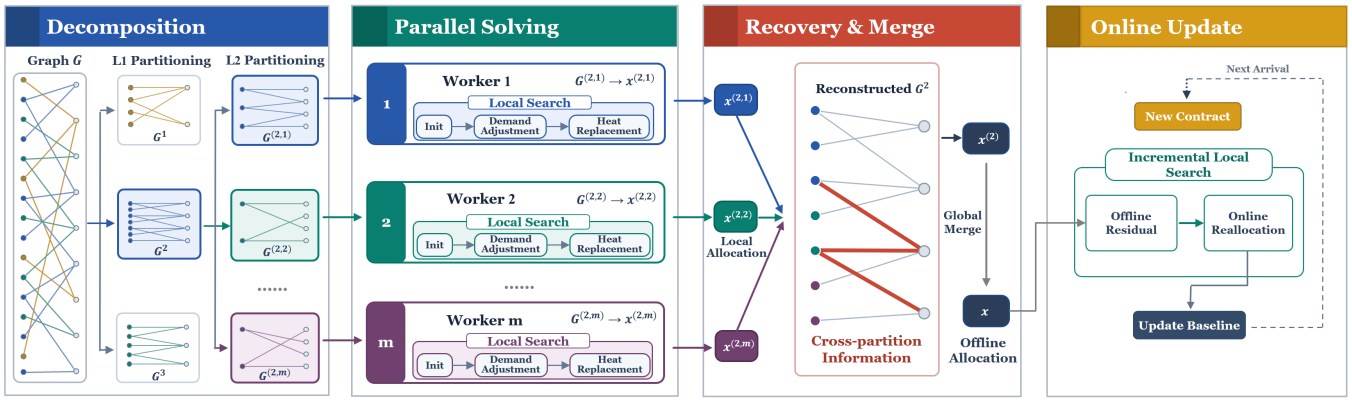


Figure 3: Overall Framework of PILLS

Proposition 1 (Slack concentration). *For any fixed $B' \subseteq B^{(s)}$, let $N(B')$ denote the supplies eligible for demand in B'. Let* $\text{slack}^{(s)}(B')$ *and* $\text{slack}^{(s,i)}(B')$ *denote the total eligible supply capacity minus the total demand of B' in $G^{(s)}$ and $G^{(s,i)}$, respectively. Under the randomized supply partition, for every $i \in \{1, \dots, m\}$,*

$$\mathbb{E}\left[\text{slack}^{(s,i)}(B')\right] = \frac{1}{m}\,\text{slack}^{(s)}(B').$$

Moreover, if $\text{slack}^{(s)}(B') > 0$, *then*

$$\Pr\left[\text{slack}^{(s,i)}(B') < 0\right] \le \exp\left(-\frac{2\,\text{slack}^{(s)}(B')^2}{m^2 \sum_{a\in N(B')} \text{capacity}(a)^2}\right).$$

Proposition 2 (Expected relative underdelivery). *Suppose that $G^{(s)}$ has an allocation $x = (x_{ab})_{(a,b)\in E^{(s)}}$ that exactly satisfies every demand. For this allocation, define $L \ge 1$ such that $1/L$ is the largest fraction of any demand contributed by a single supply: After partitioning, we restrict x to each subproblem $G^{(s,i)}$ and denote the resulting underdelivery of demand b by $u_b^{(s,i)}$. Then the expected relative underdelivery after partitioning satisfies*

$$\frac{\mathbb{E}\left[\sum_{i=1}^{m}\sum_{b\in B^{(s)}} u_b^{(s,i)}\right]}{\sum_{b\in B^{(s)}} \text{demand}(b)} \le \frac{1}{2}\sqrt{\frac{m-1}{L}}.$$

Proofs of both propositions are provided in Appendix A. The first proposition establishes an exponential lower-tail bound for local slack. In the large-market regime, where each supply contributes only a vanishing fraction of any demand, the second proposition guarantees negligible expected relative underdelivery.

3.2 Local Search for Solving Subproblems

After partitioning, we solve the resulting subproblems independently and in parallel using a local search procedure.

Each subproblem $G^{(s,i)}$ begins with an initialization step, followed by two local improvement phases. The *Demand Adjustment Phase* reallocates supply to deficit demands to reduce underdelivery, whereas the *Heat Replacement Phase* shifts allocations from high-heat to lower-heat supplies to improve allocation quality.

3.2.1 ***Initialization.*** We construct a baseline allocation by greedily processing supplies in $A^{(s,i)}$ in ascending order of heat. For each supply a, its eligible demands are considered in ascending order of degree, thereby prioritizing those with fewer eligible supplies. The available capacity of a is allocated in this order until a is exhausted or all its eligible demands are satisfied.

3.2.2 ***Demand Adjustment Phase.*** After initialization, some demands may remain unsatisfied. Meanwhile, some supplies still have residual capacity but cannot allocate it, as they are not adjacent to such unsatisfied demands. The goal of this phase is thus to redistribute such residual capacity to unsatisfied demands. To achieve this, we perform local adjustments to the allocation to further reduce deficits.

We first classify demands into unsatisfied demands $D^{\text{unsat}} = \left\{b \in B^{\text{off}} \;\middle|\; \sum_{a\in A} M_{ab}x_{ab} < \text{demand}(b)\right\}$, and satisfied demands $D^{\text{sat}} = B^{\text{off}} \setminus D^{\text{unsat}}$. For each $b \in D^{\text{unsat}}$, let $q_b = \text{demand}(b) - \sum_{a\in A} M_{ab}x_{ab}$ denote its current deficit. Correspondingly, supplies are divided into those adjacent to at least one unsatisfied demand, $S^{\text{adj}} = \left\{a \in A \;\middle|\; \exists b \in D^{\text{unsat}} : (a,b) \in E\right\}$, and the remaining supplies $S^{\text{rem}} = A \setminus S^{\text{adj}}$. Let r_a denote the residual capacity of supply a.

Consider a satisfied demand $b_0 \in D^{\text{sat}}$ that receives allocation $x_{a_1b_0}$ from a supply $a_1 \in S^{\text{adj}}$, and suppose that an eligible supply $a_0 \in S^{\text{rem}}$ has residual capacity. Supply a_0 may take over part of the allocation currently provided by a_1 to b_0, thereby releasing an equal amount of capacity at a_1. The released capacity can subsequently be assigned to an unsatisfied demand $b_1 \in D^{\text{unsat}}$. We represent such a replacement by a *reallocation chain*:

$$P = (a_0, b_0, a_1, b_1) \in S^{\text{rem}} \times D^{\text{sat}} \times S^{\text{adj}} \times D^{\text{unsat}}.$$

A reallocation chain $P = (a_0, b_0, a_1, b_1)$ is admissible if

$$(a_0, b_0), (a_1, b_0), (a_1, b_1) \in E, \quad \text{and } r_{a_0} > 0,\ x_{a_1b_0} > 0,\ q_{b_1} > 0.$$

The local reallocation along P is defined as:

$$\delta(P) = \min\left\{r_{a_0}, x_{a_1b_0}, q_{b_1}\right\}.$$

The corresponding reallocation updates the allocation as follows:

$$x_{a_0b_0} \leftarrow x_{a_0b_0} + \delta(P),\ \ x_{a_1b_0} \leftarrow x_{a_1b_0} - \delta(P),\ \ x_{a_1b_1} \leftarrow x_{a_1b_1} + \delta(P).$$

This phase repeatedly applies an admissible reallocation chain until no such chain exists. In each reallocation, the allocation removed from a demand in D^{sat} is replaced by the same amount from

a supply in S^{rem}. Hence, all previously satisfied demands remain satisfied, while the total deficit over D^{unsat} decreases by $\delta(P)$.

In addition, we establish a worst-case theoretical guarantee on deficit reduction relative to the optimum over admissible chains.

Proposition 3.1 (Approximation guarantee for demand adjustment). *Let Δ denote the total deficit reduction achieved by the demand adjustment phase. Let $\Delta^\star$ denote the maximum total deficit reduction achievable from the same initial allocation by applying admissible reallocation chains. Then*

$$\Delta \geq \frac{1}{3}\Delta^\star.$$

The detailed proof is provided in Appendix B.

3.2.3 ***Heat Replacement Phase.*** After the demand adjustment phase, the resulting allocation may still use higher-heat supplies when lower-heat alternatives are available. This phase shifts allocations from higher-heat supplies to lower-heat supplies while preserving the total allocation of each demand. To achieve this, we apply a heat-based local reallocation procedure, as detailed below.

Consider a lower-heat supply a_0 with residual capacity that is eligible for a demand $b \in B$ currently receiving allocation $x_{a_1 b}$ from a higher-heat supply a_1. Supply a_0 may take over part of the allocation provided by a_1 to b. We represent such a replacement by

$$P = (a_0, b, a_1) \in A \times B \times A,$$

where

$$(a_0, b), (a_1, b) \in E, \qquad r_{a_0} > 0, \qquad x_{a_1 b} > 0, \qquad w_{a_0} < w_{a_1}.$$

The local replacement amount is defined as

$$\delta(P) = \min\{r_{a_0}, x_{a_1 b}\}.$$

Applying P changes the objective by

$$\Delta f(P) = 2\delta(P)\big(w_{a_1} r_{a_1} - w_{a_0} r_{a_0}\big) + \delta(P)^2\big(w_{a_0} + w_{a_1}\big).$$

A heat replacement P is admissible if

$$\Delta f(P) > 0.$$

The corresponding allocation variables are updated by

$$x_{a_0 b} \leftarrow x_{a_0 b} + \delta(P), \qquad x_{a_1 b} \leftarrow x_{a_1 b} - \delta(P).$$

The residual capacities are updated accordingly as

$$r_{a_0} \leftarrow r_{a_0} - \delta(P), \qquad r_{a_1} \leftarrow r_{a_1} + \delta(P).$$

At each iteration, we apply an admissible replacement with the largest objective improvement $\Delta f(P)$. The phase terminates when no admissible replacement remains. Each replacement shifts part of demand b's allocation from the higher-heat supply a_1 to the lower-heat supply a_0, preserving all demand totals while reducing the allocation from the higher-heat supply.

We further establish a local optimality guarantee for the allocation obtained after the heat replacement phase.

Proposition 3.2 (Local optimality of heat replacement). *At the end of the heat replacement phase, for every heat replacement P and every $0 < \eta \leq \delta(P)$, reallocating η units along P cannot improve the offline objective.*

The detailed proof is provided in Appendix C.

3.3 Merging via Cross-Subproblem Recovery

After all partitioned subproblems have been solved independently by local search, we merge their local allocations within each first-level subgraph. Since the supply sets of the m second-level subproblems are disjoint, the merged allocation on $G^{(s)}$ satisfies

$$\sum_{a \in A^{(s)}} x_{ab}^{(s)} = \sum_{i=1}^{m} \sum_{a \in A^{(s,i)}} x_{ab}^{(s,i)}, \qquad \forall b \in B^{(s)},$$

where $x_{ab}^{(s,i)}$ is the allocation obtained for subproblem $G^{(s,i)}$, and $x_{ab}^{(s)}$ is the resulting allocation on $G^{(s)}$. Based on the merged allocation, q_b and r_a are recomputed accordingly.

While Proposition 2 shows that supply-side partitioning yields negligible expected relative underdelivery in the large-market regime, real-world allocations may exhibit different levels of supply concentration across demands. When a few supplies contribute a substantial fraction of a demand, its eligible supplies may be unevenly distributed among the subproblems, leaving the demand unsatisfied in local allocations. We therefore perform a cross-subproblem adjustment as a safeguard against such partition-induced imbalance.

The cross-subproblem adjustment first assigns residual capacity directly to unsatisfied demands. Specifically, for any $(a, b) \in E^{(s)}$ with $r_a > 0$ and $q_b > 0$, we allocate

$$\delta(a, b) = \min\{r_a, q_b\}$$

and update

$$x_{ab}^{(s)} \leftarrow x_{ab}^{(s)} + \delta(a, b), \quad r_a \leftarrow r_a - \delta(a, b), \quad q_b \leftarrow q_b - \delta(a, b).$$

This process is repeated until no such edge remains. After the process terminates, we then apply the demand adjustment phase to each $G^{(s)}$. The same adjustment operations are then performed on the complete $G^{(s)}$, recovering the information lost during supply partitioning and allowing reallocation chains to involve supplies from different second-level subproblems.

Moreover, we establish a worst-case theoretical guarantee on the total allocation obtained after the cross-subproblem adjustment.

Proposition 3.3 (Total-allocation guarantee after cross–subproblem recovery). *Let Δ denote the total allocation in the final solution obtained on $G^{(s)}$ after cross-subproblem recovery, and let $\Delta^\star$ denote the maximum total allocation achievable by any feasible allocation on $G^{(s)}$ that contains no overdelivery. Then*

$$\Delta \geq \frac{2}{3}\Delta^\star.$$

The detailed proof is provided in Appendix D.

Finally, since different first-level subgraphs have no connecting edges, their adjusted allocations can be directly combined to obtain the final allocation:

$$x_{ab} = x_{ab}^{(s)}, \qquad \forall (a, b) \in E^{(s)},\ s \in \{1, \ldots, k\},$$

3.4 Online Update for New Contracts

In the online stage, new contracts arrive sequentially, and the allocation available to each arriving contract must be determined in real time. Re-solving the entire allocation problem after every arrival becomes impractical for large-scale allocation. Instead, we take the

offline solution as the baseline and design an incremental update method that locally adjusts it for each newly arrived contract.

Consider a newly arrived contract b_{new}. We first assign residual capacity directly to b_{new}. Specifically, for any $(a, b_{\text{new}}) \in E$ with $r_a > 0$, we update

$$x_{ab_{\text{new}}} \leftarrow x_{ab_{\text{new}}} + r_a, \qquad r_a \leftarrow 0.$$

We then construct online reallocation chains following the same alternating structure as those in the demand adjustment phase. Specifically, for the current arrival b_{new}, let $S^{\text{adj}} = \{a \in A : (a, b_{\text{new}}) \in E\}$ denote its adjacent supplies and let $S^{\text{rem}} = A \setminus S^{\text{adj}}$. An online reallocation chain is represented by

$$P = (a_0, b_0, a_1, b_{\text{new}}) \in S^{\text{rem}} \times \left(B^{\text{off}} \cup B^{\text{on}}_{\text{prev}}\right) \times S^{\text{adj}} \times \{b_{\text{new}}\}.$$

An online reallocation $P = (a_0, b_0, a_1, b_{\text{new}})$ is admissible if

$$(a_0, b_0), (a_1, b_0), (a_1, b_{\text{new}}) \in E, \qquad r_{a_0} > 0, \qquad x_{a_1 b_0} > 0.$$

The reallocation amount is

$$\delta(P) = \min\left\{r_{a_0}, x_{a_1 b_0}\right\}.$$

The corresponding allocation variables are updated by

$$x_{a_0 b_0} \leftarrow x_{a_0 b_0} + \delta(P), \;\; x_{a_1 b_0} \leftarrow x_{a_1 b_0} - \delta(P), \;\; x_{a_1 b_{\text{new}}} \leftarrow x_{a_1 b_{\text{new}}} + \delta(P).$$

In online execution, the procedure performs as many admissible reallocations as allowed by the response-time limit and terminates when the limit is reached.

Finally, for any online arrival sequence, we establish a worst-case theoretical guarantee for the total allocation relative to the optimum obtained by re-solving all offline and arrived online contracts.

Proposition 3.4 (Incremental allocation guarantee relative to reoptimization). *Let $b^{\text{on}}_1, \ldots, b^{\text{on}}_t$ denote the first t online contracts indexed by their arrival order, and define $B_t = B^{\text{off}} \cup \{b^{\text{on}}_1, \ldots, b^{\text{on}}_t\}$. Let A_t denote the total allocation maintained after processing these online contracts, and let $A^\star_t$ denote the maximum total allocation obtainable by re-solving the entire allocation problem for all contracts in B_t without overdelivering any offline contract. For each $\tau \le t$, let R_τ denote the maximum amount that can still be allocated to b^{on}_τ through the admissible chains remaining when its processing stops. Then*

$$A_t \ge \frac{2}{3}\left(A^\star_t - \sum_{\tau=1}^{t} R_\tau\right).$$

In particular, if every arrival is processed until no admissible chain remains, then

$$A_t \ge \frac{2}{3} A^\star_t.$$

The detailed proof is provided in Appendix E.

4 Experiment

We have implemented PILLS and deployed it in the Alibaba GD advertising system for one year. Comprehensive evaluations and deployment performance demonstrate that PILLS consistently improves allocation quality, delivery rate, revenue, and CPM.

4.1 Experiment Settings

4.1.1 Implementation and Setup. PILLS is programmed in C++, compiled by g++ with '-O3' option. All experiments are run on a server with an AMD EPYC 9654 CPU, 1024 GB RAM, and Ubuntu 20.04.4. The penalty coefficient p is set to 10,000. The heat is derived from historical traffic statistics, and its distribution across the evaluated datasets is reported in Appendix F. The default number of parallel CPU cores is 32. The datasets, heat configurations, and source code are available in our repository[2].

4.1.2 Datasets. The datasets come from real GD advertising allocation at Alibaba. We select ten daily datasets from regular days between April 16 and May 16, 2025, and promotion periods between May 31 and June 30, 2025, each containing 100 online allocation requests. The datasets cover diverse targeting types (e.g., gender, age, income, and interests), markets across multiple application domains within Alibaba (e.g., logistics, social networking, navigation, and e-commerce), 422 geographic regions, multiple device types (e.g., iOS, Android, and PC), and user behaviors reflected by clicks and search queries.

Dataset	Supply	Demand	Dataset	Supply	Demand
$\mathcal{D}_1$	32,505,062	330	$\mathcal{D}_2$	87,263,918	1021
$\mathcal{D}_3$	99,824,983	1023	$\mathcal{D}_4$	73,662,802	1019
$\mathcal{D}_5$	74,228,133	1020	$\mathcal{D}_6$	78,860,455	1021
$\mathcal{D}_7$	79,975,731	1018	$\mathcal{D}_8$	77,864,997	1019
$\mathcal{D}_9$	81,020,881	1023	$\mathcal{D}_{10}$	86,934,221	1022

Table 1: Description of the datasets

Table 1 reports the numbers of supply and demand nodes in each dataset. The scale reaches nearly 100 million supply nodes, reflecting the scale of GD allocation in production. Detailed dataset statistics, including the supply–demand ratio, sparsity, and node degrees, are provided in Table 9 in Appendix F.

4.2 Offline Allocation Performance

4.2.1 ***Metrics and Competitors.*** In the offline stage, three metrics are evaluated under a one-hour time limit:

- **#Ud:** the underdelivery rate, defined as $\#\text{Ud} = \frac{\sum_b u_b}{\sum_b \text{demand}(b)}$.
- **#Obj:** the obtained offline objective value f_{off}.
- **#Gap:** the relative gap to the best-known objective $f^{\text{best}}_{\text{off}}$,[3] defined as $\#\text{Gap} = \frac{f^{\text{best}}_{\text{off}} - f_{\text{off}}}{|f^{\text{best}}_{\text{off}}|}$.

We compare PILLS with the following offline competitors:

We compare PILLS with the following offline competitors:

- **HWM** [5]: an effective greedy GD allocation heuristic, using 32 threads to parallelize per-contract supply-side computations.
- **Gurobi (v12.0)** [14]: a top-performing commercial mathematical optimization solver [26], evaluated in exact (Gurobi_exact) and heuristic (Gurobi_heur) modes, as well as with HWM solutions as warm starts (Gurobi_WS), using 32 CPU threads.

[2] https://anonymous.4open.science/r/PILLS-C1F0

[3] Gurobi relies on branch-and-bound-based methods, which can certify optimality but have exponential worst-case complexity and may require much longer than the practical one-hour budget at our scale. We therefore run Gurobi for one month per instance to obtain a high-quality reference solution.

	Gurobi_exact			*Gurobi_heur*			*Gurobi_WS*			*HWM*			*Rap*			*PILLS*		
	#Obj	*#Ud*	*#Gap*	*#Obj*	*#Ud*	*#Gap*	*#Obj*	*#Ud*	*#Gap*	*#Obj*	*#Ud*	*#Gap*	*#Obj*	*#Ud*	*#Gap*	*#Obj*	*#Ud*	*#Gap*
$\mathcal{D}_1$	8.01×10^{15}	1.00	25.1%	8.01×10^{15}	1.00	25.1%	9.85×10^{15}	0.300	7.8%	9.81×10^{15}	0.316	8.2%	8.29×10^{15}	0.887	22.4%	$\mathbf{1.07\times10^{16}}$	**0.000**	**0.19%**
$\mathcal{D}_2$	-3.84×10^{12}	1.00	385.2%	-3.84×10^{12}	1.00	385.2%	9.10×10^{11}	0.069	32.5%	8.89×10^{11}	0.071	34.1%	-2.74×10^{12}	0.794	303.1%	$\mathbf{1.31\times10^{12}}$	**0.029**	**2.88%**
$\mathcal{D}_3$	-3.74×10^{12}	1.00	364.9%	-3.74×10^{12}	1.00	364.9%	9.91×10^{11}	0.070	29.8%	9.69×10^{11}	0.072	31.4%	-2.63×10^{12}	0.796	286.5%	$\mathbf{1.41\times10^{12}}$	**0.024**	**0.38%**
$\mathcal{D}_4$	-3.49×10^{12}	1.00	307.8%	-3.49×10^{12}	1.00	307.8%	1.27×10^{12}	0.060	24.2%	1.25×10^{12}	0.062	25.5%	-1.77×10^{12}	0.671	205.0%	$\mathbf{1.68\times10^{12}}$	**0.025**	**0.11%**
$\mathcal{D}_5$	-3.65×10^{12}	1.00	334.9%	-3.65×10^{12}	1.00	334.9%	1.13×10^{12}	0.065	27.2%	1.11×10^{12}	0.067	28.6%	-2.53×10^{12}	0.792	262.9%	$\mathbf{1.54\times10^{12}}$	**0.030**	**1.08%**
$\mathcal{D}_6$	-3.70×10^{12}	1.00	338.5%	-3.70×10^{12}	1.00	338.5%	1.15×10^{12}	0.063	25.9%	1.13×10^{12}	0.065	27.3%	-2.60×10^{12}	0.796	267.5%	$\mathbf{1.55\times10^{12}}$	**0.028**	**0.15%**
$\mathcal{D}_7$	-3.69×10^{12}	1.00	340.7%	-3.69×10^{12}	1.00	340.7%	1.09×10^{12}	0.061	29.0%	1.07×10^{12}	0.063	30.3%	-2.53×10^{12}	0.782	264.9%	$\mathbf{1.48\times10^{12}}$	**0.026**	**3.50%**
$\mathcal{D}_8$	-3.68×10^{12}	1.00	347.5%	-3.68×10^{12}	1.00	347.5%	1.08×10^{12}	0.066	27.2%	1.06×10^{12}	0.068	28.6%	-2.61×10^{12}	0.799	275.6%	$\mathbf{1.47\times10^{12}}$	**0.030**	**0.72%**
$\mathcal{D}_9$	-3.77×10^{12}	1.00	362.2%	-3.77×10^{12}	1.00	362.2%	1.03×10^{12}	0.066	28.1%	1.01×10^{12}	0.068	29.6%	-2.68×10^{12}	0.796	286.1%	$\mathbf{1.44\times10^{12}}$	**0.028**	**0.17%**
$\mathcal{D}_{10}$	-3.92×10^{12}	1.00	394.5%	-3.92×10^{12}	1.00	394.5%	9.21×10^{11}	0.068	30.9%	9.00×10^{11}	0.070	32.5%	-2.81×10^{12}	0.794	310.9%	$\mathbf{1.32\times10^{12}}$	**0.029**	**0.97%**
Avg.	7.98×10^{14}	1.00	320.1%	7.98×10^{14}	1.00	320.1%	9.86×10^{14}	0.089	26.3%	9.82×10^{14}	0.092	27.6%	8.27×10^{14}	0.791	248.5%	$\mathbf{1.07\times10^{15}}$	**0.025**	**1.02%**

Table 2: Offline comparison of #Ud, #Obj, and #Gap with state-of-the-art GD advertising algorithms.

	Gurobi_WS-FIFO		*Gurobi_WS-PILLS*		*HWM-FIFO*		*HWM-PILLS*		*Rap-Rap*		*PILLS-FIFO*		*PILLS-PILLS*	
	#Top	*#Obj*	*#Top*	*#Obj*	*#Top*	*#Obj*	*#Top*	*#Obj*	*#Top*	*#Obj*	*#Top*	*#Obj*	*#Top*	*#Obj*
$\mathcal{D}_1$	0	3.65×10^{8}	0	3.73×10^{8}	0	3.62×10^{8}	0	3.70×10^{8}	0	2.25×10^{8}	0	4.27×10^{8}	**100**	$\mathbf{4.35\times10^{8}}$
$\mathcal{D}_2$	0	9.70×10^{8}	0	1.14×10^{9}	0	9.59×10^{8}	0	1.13×10^{9}	0	8.00×10^{8}	0	1.18×10^{9}	**100**	$\mathbf{1.35\times10^{9}}$
$\mathcal{D}_3$	0	1.41×10^{9}	0	1.48×10^{9}	0	1.40×10^{9}	0	1.47×10^{9}	0	1.15×10^{9}	0	1.64×10^{9}	**100**	$\mathbf{1.72\times10^{9}}$
$\mathcal{D}_4$	0	1.38×10^{9}	0	1.46×10^{9}	0	1.37×10^{9}	0	1.45×10^{9}	0	1.10×10^{9}	0	1.55×10^{9}	**100**	$\mathbf{1.64\times10^{9}}$
$\mathcal{D}_5$	0	1.29×10^{9}	0	1.40×10^{9}	0	1.28×10^{9}	0	1.39×10^{9}	0	1.01×10^{9}	0	1.47×10^{9}	**100**	$\mathbf{1.58\times10^{9}}$
$\mathcal{D}_6$	0	1.41×10^{9}	0	1.47×10^{9}	0	1.40×10^{9}	0	1.46×10^{9}	0	1.13×10^{9}	0	1.59×10^{9}	**100**	$\mathbf{1.65\times10^{9}}$
$\mathcal{D}_7$	0	1.07×10^{9}	0	1.20×10^{9}	0	1.06×10^{9}	0	1.19×10^{9}	0	9.00×10^{8}	0	1.25×10^{9}	**100**	$\mathbf{1.38\times10^{9}}$
$\mathcal{D}_8$	0	1.24×10^{9}	0	1.37×10^{9}	0	1.23×10^{9}	0	1.36×10^{9}	0	1.04×10^{9}	0	1.42×10^{9}	**100**	$\mathbf{1.54\times10^{9}}$
$\mathcal{D}_9$	0	1.04×10^{9}	0	1.18×10^{9}	0	1.03×10^{9}	0	1.17×10^{9}	0	8.70×10^{8}	0	1.23×10^{9}	**100**	$\mathbf{1.38\times10^{9}}$
$\mathcal{D}_{10}$	0	1.18×10^{9}	0	1.31×10^{9}	0	1.17×10^{9}	0	1.30×10^{9}	0	9.90×10^{8}	0	1.38×10^{9}	**100**	$\mathbf{1.51\times10^{9}}$
Avg.	0	1.14×10^{9}	0	1.24×10^{9}	0	1.13×10^{9}	0	1.23×10^{9}	0	9.22×10^{8}	0	1.31×10^{9}	**100**	$\mathbf{1.42\times10^{9}}$

Table 3: Online comparison of #Top and #Obj with GD advertising algorithms.

- **Rap** [31]: a representative primal–dual algorithm for GD, using its parallel implementation with 32 CPU threads.

4.2.2 ***Results.*** Table 2 shows that PILLS produces the best overall offline allocations within the time limit. On the smaller dataset $\mathcal{D}_1$, all algorithms achieve competitive #Obj values. As the problem scale increases, however, the advantage of PILLS becomes more pronounced: it achieves substantially lower #Ud and higher #Obj than the competing methods on the larger datasets $\mathcal{D}_2$–$\mathcal{D}_{10}$. Moreover, PILLS achieves the smallest #Gap on all datasets, averaging only 1.02%, whereas other algorithms exceed 26% and even 320% [4].

4.3 Online Allocation Performance

4.3.1 ***Metrics and Competitors.*** Three metrics are used for comparison on 100 online requests, with a time limit of 60 seconds:

- **#Top:** the number of online contracts for which the method delivers the most impressions among all competitors.
- **#Obj:** the online allocation objective, excluding underdelivery from offline contracts, defined as $\sum_{b\in B^{\text{on}}}\sum_{a\in A} M_{ab}x_{ab} - p\sum_{b\in B^{\text{on}}} u_b$.

We compare PILLS with the following online competitors:

- **FIFO**: processes online contracts in arrival order and allocates to each contract all residual capacity from its eligible supply nodes.
- **Rap** [31]: An online primal–dual algorithm that uses its offline allocation as the baseline and updates it upon each arrival.

[4]Within the one-hour time limit, Gurobi without warm starts found only the trivial solution with $x_{ab} = 0$ and $u_b = \text{demand}(b)$ for all contracts, resulting in #Ud = 1.

Each online algorithm is evaluated on offline allocations produced by different methods, yielding all possible offline–online combinations (For Gurobi, we use its best-performing offline configuration, Gurobi_WS), as follows:

- **Gurobi_WS–FIFO**: offline from Gurobi_WS, online by FIFO.
- **Gurobi_WS–PILLS**: offline from Gurobi_WS, online by PILLS.
- **HWM–FIFO**: offline from HWM, online by FIFO.
- **HWM–PILLS**: offline from HWM, online by PILLS.
- **Rap–Rap**: both offline and online by Rap.
- **PILLS–FIFO**: offline from PILLS, online by FIFO.
- **PILLS–PILLS**: both offline and online by PILLS.

4.3.2 ***Results.*** Table 3 demonstrates the advantages of PILLS in both offline allocation and online updating. First, with the online strategy fixed, the offline allocation produced by PILLS substantially improves subsequent online allocation quality over other offline methods. Second, with the offline allocation fixed, the PILLS online algorithm achieves the best performance across all metrics. Combining these advantages, PILLS–PILLS delivers the most impressions to every new contract and achieves the highest #Obj.

4.4 End-to-End Business Performance

4.4.1 ***Business Performance Comparison.*** We evaluate the production business performance of different allocation pipelines in our GD advertising system using ten industrial datasets. We evaluate three business metrics:

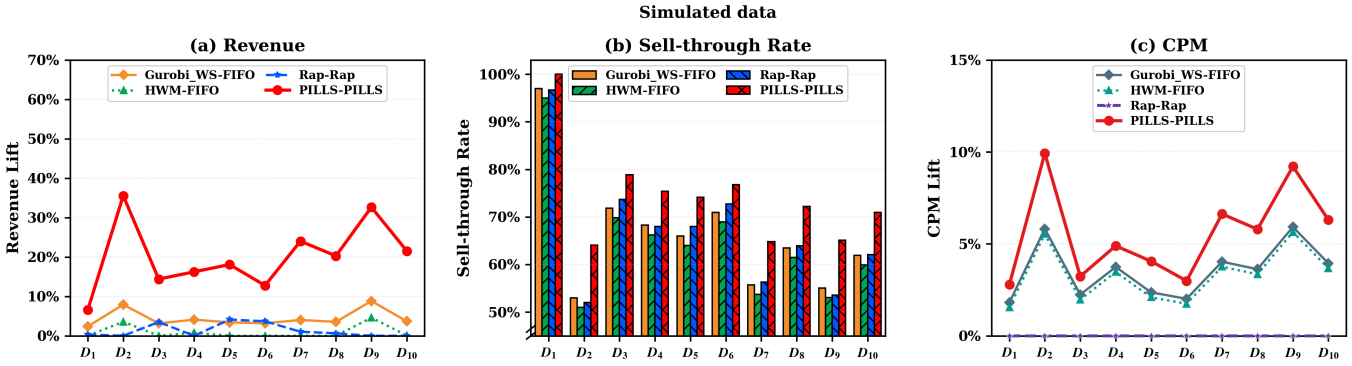


Figure 4: End-to-End Business Performance.

Table 4: Scale of the production A/B test

	Weak1 May 29–Jun 4	Weak2 Jun 5–11	Weak3 Jun 12–18	Weak4 Jun 19–25
#Impressions	668M	637M	778M	494M
#Contracts	7,143	7,117	7,265	5,993

Table 5: Business improvements over Gurobi_WS-FIFO

Metric	Week1	Week2	Week3	Week4	Avg	95% CI
Revenue	+7.1%	+8.8%	+8.5%	+4.7%	**+7.28%**	[4.21, 10.39]%
#SR	+4.12%	+5.08%	+4.47%	+2.63%	**+4.08%**	[2.47, 5.45]%
CPM	+2.86%	+3.54%	+3.86%	+2.02%	**+3.07%**	[1.75, 4.26]%

- **Revenue:** the total revenue, computed as the sum of the revenue generated by the impressions allocated to each demand.
- **Sell-through Rate (#SR):** defined as $\frac{\sum_{a\in A}\sum_{b\in B^{\text{off}}\cup B^{\text{on}}} M_{ab}x_{ab}}{\sum_{a\in A}\text{capacity}(a)}$.
- **CPM:** the revenue generated per thousand allocated impressions, defined as $\text{CPM} = \frac{\text{Revenue}}{\sum_{a\in A}\sum_{b\in B^{\text{off}}\cup B^{\text{on}}} M_{ab}x_{ab}} \times 1000$.

We compare the complete PILLS–PILLS offline–online pipeline with three existing GD allocation pipelines: Gurobi_WS–FIFO, HWM–FIFO, and Rap–Rap. Revenue and CPM are normalized as percentage improvements over their respective minima across pipelines on each dataset, while Sell-through Rate is reported directly. Figure 4 shows that PILLS–PILLS achieves the best overall business performance, demonstrating substantial practical business gains.

4.4.2 ***Online A/B Test.*** We conduct a four-week production A/B test in our GD advertising system from May 29 to June 25, 2025, comparing PILLS–PILLS against Gurobi_WS-FIFO, the strongest baseline identified above.

Each day, incoming user traffic and GD contracts are randomly assigned to two mutually exclusive groups, with comparable supply volumes and contract characteristics, including demand volumes, bid prices and targeting characteristics. Each group independently executes its allocation pipeline using the original contract demands, arrival order, and advertisers' actual bids.

Table 5 reports business improvements over Gurobi_WS-FIFO, along with statistical significance using 95% confidence intervals. PILLS–PILLS consistently outperforms baseline across all four weeks, delivering a **7.28% average Revenue improvement**, along with improvements of 4.08% in Sell-through Rate and 3.07% in CPM.

4.5 Serving Performance

Although serving is not our focus, we evaluate the impact of GD allocation solutions on serving performance, using a practical serving procedure for PILLS and the respective serving strategy for each competitor. Details and results are in Appendix G. The results show that the high-quality GD allocations produced by PILLS lead to the best serving performance among all compared methods.

4.6 Evaluation on Public Datasets

Due to commercial sensitivity, publicly available GD allocation datasets are scarce. To the best of our knowledge, no publicly available dataset previously supported billion-scale experiments. We identified only three publicly available datasets [6, 19, 21]. These datasets contain only 20,735–115,050 supply users, far below the billion-scale addressed in our work. We nevertheless include them in our evaluation to examine the generalizability of PILLS.

On these smaller instances, competing methods achieve much better performance than on billion-scale problems, highlighting their scalability limitations. Nevertheless, PILLS still achieves the best performance. Detailed results are provided in Appendix I.

4.7 Comprehensive Analysis of our Algorithm

We further examine PILLS from several perspectives.

- **Ablation Study.** We conduct an ablation study by removing five key components of PILLS. Table 7 reports the #Top results over 1,000 online requests, confirming their effectiveness.
- **Parallel Speedup.** Figure 5 shows that PILLS achieves near-linear speedup up to 16 threads in offline stage, reducing solving time from 3102s to 309s and further to 176s with 64 threads.

Table 6: Online allocation drift relative to offline reoptimization

#Requests	1	5	10	20	30	40	50	60	80	100
Drift (%)	0.05	0.08	0.31	0.46	0.57	0.66	0.72	0.80	0.95	1.15

Table 7: Ablation results in terms of #Top over online requests

Removed Component	PILLS #Top (Margin)	Variant #Top
Demand Adjustment	**781 (+562)**	219
Heat Replacement	**743 (+486)**	257
Supply-Side Partitioning	**905 (+810)**	95
Cross-Subproblem Recovery	**721 (+442)**	279
Online Reallocation	**1000 (+1000)**	0

- **Independent Subgraph Analysis.** We analyze the number and sizes of independent subgraphs by Level-1 division across datasets, with detailed results provided in Appendix H.
- **Online Allocation Drift.** We assess online allocation quality by comparing cumulative allocation with offline PILLS reoptimization. For the first k online requests, let $A_{\text{on}}(k)$ denote the cumulative allocation achieved by online PILLS, and $A_{\text{off}}(k)$ denote the allocation obtained by offline PILLS, which jointly reoptimizes all historical contracts and the first k online contracts. We define the drift as $\text{Drift}(k) = 1 - A_{\text{on}}(k)/A_{\text{off}}(k)$. Table 6 reports the average drift across ten industrial datasets, which increases gradually to only 1.15% after 100 requests.
- **Runtime Analysis.** We analyze the computational cost of each phase in PILLS. Table 8 reports the average runtime and its proportion in each stage.
- **Memory.** Across the ten datasets, the memory usage of PILLS ranges from 101 to 167 GB, with a peak usage of 238 GB.

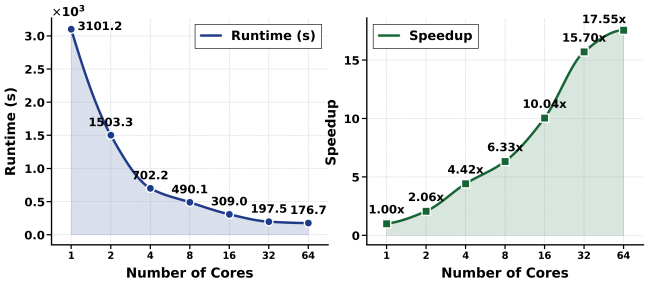


Figure 5: Parallel Performance Analysis

5 Conclusion and Future Work

In this work, we present PILLS, an end-to-end framework for billion-scale GD advertising inventory allocation. PILLS combines an offline decompose–solve–recover pipeline with online incremental updates, enabling high-quality allocation within practical business time limits. Billion-scale experiments in Alibaba GD systems demonstrate its effectiveness and improvements in delivery rate, revenue, and CPM. In future work, we will extend PILLS to support more complex business constraints in real-world advertising platforms.

Table 8: Phase-wise runtime analysis of PILLS

Stage	Phase	Time (s)	Share (%)
Offline	Decomposition	15.8	8.0
	Initialization	20.6	10.4
	Demand Adjustment	92.4	46.8
	Heat Replacement	45.9	23.2
	Merge & Recovery	22.8	11.5
Online	Residual Allocation	3.4	5.6
	Reallocation	54.2	90.3
	State Update	2.3	3.8

References

[1] Shipra Agrawal and Nikhil R Devanur. 2014. Fast algorithms for online stochastic convex programming. In *Proceedings of the twenty-sixth annual ACM-SIAM symposium on Discrete algorithms*. SIAM, 1405–1424.

[2] Shipra Agrawal, Zizhuo Wang, and Yinyu Ye. 2014. A dynamic near-optimal algorithm for online linear programming. *Operations Research* 62, 4 (2014), 876–890.

[3] Anand Bhalgat, Jon Feldman, and Vahab Mirrokni. 2012. Online allocation of display ads with smooth delivery. In *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1213–1221.

[4] Vijay Bharadwaj, Peiji Chen, Wenjing Ma, Chandrashekhar Nagarajan, John Tomlin, Sergei Vassilvitskii, Erik Vee, and Jian Yang. 2012. Shale: an efficient algorithm for allocation of guaranteed display advertising. In *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1195–1203.

[5] Peiji Chen, Wenjing Ma, Srinath Mandalapu, Chandrashekhar Nagarjan, Jayavel Shanmugasundaram, Sergei Vassilvitskii, Erik Vee, Manfai Yu, and Jason Zien. 2012. Ad serving using a compact allocation plan. In *Proceedings of the 13th ACM Conference on Electronic Commerce*. 319–336.

[6] Xiao Cheng, Chuanren Liu, Liang Dai, Peng Zhang, Zhen Fang, and Zhonglin Zu. 2022. An Adaptive Unified Allocation Framework for Guaranteed Display Advertising. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*. 132–140.

[7] Liang Dai, Zhonglin Zu, Hao Wu, Liang Wang, and Bo Zheng. 2023. Fairness-aware Guaranteed Display Advertising Allocation under Traffic Cost Constraint. In *Proceedings of the ACM Web Conference 2023*. 3572–3580.

[8] Nikhil R Devanur, Kamal Jain, Balasubramanian Sivan, and Christopher A Wilkens. 2011. Near optimal online algorithms and fast approximation algorithms for resource allocation problems. In *Proceedings of the 12th ACM conference on Electronic commerce*. 29–38.

[9] Zhen Fang, Yang Li, Chuanren Liu, Wenxiang Zhu, Yu Zheng, and Wenjun Zhou. 2019. Large-scale personalized delivery for guaranteed display advertising with real-time pacing. In *2019 IEEE International Conference on Data Mining (ICDM)*. IEEE, 190–199.

[10] Jon Feldman, Monika Henzinger, Nitish Korula, Vahab S Mirrokni, and Cliff Stein. 2010. Online stochastic packing applied to display ad allocation. In *European Symposium on Algorithms*. Springer, 182–194.

[11] Jon Feldman, Aranyak Mehta, Vahab Mirrokni, and Shan Muthukrishnan. 2009. Online stochastic matching: Beating 1-1/e. In *2009 50th Annual IEEE Symposium on Foundations of Computer Science*. IEEE, 117–126.

[12] Paul W Goldberg. 2004. Bounds for the convergence rate of randomized local search in a multiplayer load-balancing game. In *Proceedings of the twenty-third annual ACM symposium on principles of distributed computing*. 131–140.

[13] Jun Gu. 1992. Efficient local search for very large-scale satisfiability problems. *ACM SIGART Bulletin* 3, 1 (1992), 8–12.

[14] LLC Gurobi Optimization. 2022. Gurobi Optimizer Reference Manual (Gurobi Optimization, LLC). (2022).

[15] Bernhard Haeupler, Vahab S Mirrokni, and Morteza Zadimoghaddam. 2011. Online stochastic weighted matching: Improved approximation algorithms. In *Internet and Network Economics: 7th International Workshop, WINE 2011, Singapore, December 11-14, 2011. Proceedings 7*. Springer, 170–181.

[16] Xiang He, Wuyang Mao, Zhenghang Xu, Yuanzhe Gu, Yundu Huang, Zhonglin Zu, Liang Wang, Mengyu Zhao, and Mengchuan Zou. 2024. An Efficient Local Search Algorithm for Large GD Advertising Inventory Allocation with Multilinear Constraints. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1040–1049.

[17] Larry W Jacobs and Michael J Brusco. 1995. Note: A local-search heuristic for large set-covering problems. *Naval Research Logistics (NRL)* 42, 7 (1995),

1129–1140.

[18] Yan Li, Yundu Huang, Wuyang Mao, Furong Ye, Xiang He, Zhonglin Zu, and Shaowei Cai. 2024. Bi-Objective Contract Allocation for Guaranteed Delivery Advertising. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1691–1700.

[19] Jianxun Lian, Xiaohuan Zhou, Fuzheng Zhang, Zhongxia Chen, Xing Xie, and Guangzhong Sun. 2018. xdeepfm: Combining explicit and implicit feature interactions for recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1754–1763.

[20] Peng Lin, Mengchuan Zou, Zhihan Chen, and Shaowei Cai. 2024. ParaILP: A Parallel Local Search Framework for Integer Linear Programming with Cooperative Evolution Mechanism. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. 6949–6957.

[21] Langming Liu, Wanyu Wang, Chi Zhang, Bo Li, Hongzhi Yin, Xuetao Wei, Wenbo Su, Bo Zheng, and Xiangyu Zhao. 2025. Multi-task Offline Reinforcement Learning for Online Advertising in Recommender Systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 4635–4646.

[22] Desmond S Lun, Graham Rockwell, Nicholas J Guido, Michael Baym, Jonathan A Kelner, Bonnie Berger, James E Galagan, and George M Church. 2009. Large-scale identification of genetic design strategies using local search. *molecular systems biology* 5, 1 (2009), 296.

[23] Vahideh H Manshadi, Shayan Oveis Gharan, and Amin Saberi. 2012. Online stochastic matching: Online actions based on offline statistics. *Mathematics of Operations Research* 37, 4 (2012), 559–573.

[24] Wuyang Mao, Chuanren Liu, Yundu Huang, Zhonglin Zu, M Harshvardhan, Liang Wang, and Bo Zheng. 2023. End-to-End Inventory Prediction and Contract Allocation for Guaranteed Delivery Advertising. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1677–1686.

[25] Vahab S Mirrokni, Shayan Oveis Gharan, and Morteza Zadimoghaddam. 2012. Simultaneous approximations for adversarial and stochastic online budgeted allocation. In *Proceedings of the twenty-third annual ACM-SIAM symposium on Discrete Algorithms*. SIAM, 1690–1701.

[26] Hans Mittelmann. 2023. *Visualizations of Mittelmann benchmarks*. https://mattmilten.github.io/mittelmann-plots/

[27] Bart Selman, Henry A Kautz, Bram Cohen, et al. 1993. Local search strategies for satisfiability testing. *Cliques, coloring, and satisfiability* 26 (1993), 521–532.

[28] Karthik Sindhya, Kalyanmoy Deb, and Kaisa Miettinen. 2008. A local search based evolutionary multi-objective optimization approach for fast and accurate convergence. In *International Conference on Parallel Problem Solving from Nature*. Springer, 815–824.

[29] Karthik Sindhya, Kalyanmoy Deb, and Kaisa Miettinen. 2011. Improving convergence of evolutionary multi-objective optimization with local search: a concurrent-hybrid algorithm. *Natural Computing* 10 (2011), 1407–1430.

[30] Robert Johannes Maria Vaessens, Emile Hubertus Leonardus Aarts, and Jan Karel Lenstra. 1996. Job shop scheduling by local search. *Informs Journal on computing* 8, 3 (1996), 302–317.

[31] Hong Zhang, Lan Zhang, Lan Xu, Xiaoyang Ma, Zhengtao Wu, Cong Tang, Wei Xu, and Yiguo Yang. 2020. A request-level guaranteed delivery advertising planning: Forecasting and allocation. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2980–2988.

A Proof of Two-Level Decomposition

A.1 Proof of Proposition 3.1

Proposition A.1 (slack concentration). *For a fixed first-level subgraph $G^{(s)}$ and a fixed demand subset $B' \subseteq B^{(s)}$, let $N(B') = \{a \in A^{(s)} : \exists\, b \in B',\ M_{ab} = 1\}$ denote the supplies eligible for at least one demand in B'. Define the slack of B' in $G^{(s)}$ as*

$$\mathrm{slack}^{(s)}(B') = \sum_{a \in N(B')} \mathrm{capacity}(a) - \sum_{b \in B'} \mathrm{demand}(b).$$

For each $i \in \{1, \dots, m\}$, the corresponding local slack in $G^{(s,i)}$ is

$$\mathrm{slack}^{(s,i)}(B') = \sum_{a \in N(B') \cap A^{(s,i)}} \mathrm{capacity}(a) - \frac{1}{m} \sum_{b \in B'} \mathrm{demand}(b).$$

Under the supply partition described above, for every $i \in \{1, \dots, m\}$,

$$\mathbb{E}\left[\mathrm{slack}^{(s,i)}(B')\right] = \frac{\mathrm{slack}^{(s)}(B')}{m}.$$

Moreover, if $\mathrm{slack}^{(s)}(B') > 0$, then

$$\Pr\left[\mathrm{slack}^{(s,i)}(B') < 0\right] \le \exp\left(-\frac{2\,\mathrm{slack}^{(s)}(B')^2}{m^2 \sum_{a \in N(B')} \mathrm{capacity}(a)^2}\right).$$

Proof. Fix $i \in \{1, \dots, m\}$. For each $a \in N(B')$, let

$$Z_a = \begin{cases} 1, & a \in A^{(s,i)}, \\ 0, & a \notin A^{(s,i)}. \end{cases}$$

Since $A^{(s,i)}$ is sampled uniformly without replacement, $\mathbb{E}[Z_a] = 1/m$. Therefore,

$$\begin{aligned}\mathbb{E}\left[\mathrm{slack}^{(s,i)}(B')\right] &= \sum_{a \in N(B')} \mathrm{capacity}(a)\mathbb{E}[Z_a] - \frac{1}{m}\sum_{b \in B'} \mathrm{demand}(b) \\ &= \frac{\mathrm{slack}^{(s)}(B')}{m}.\end{aligned}$$

The global and local slack definitions further give

$$\mathrm{slack}^{(s,i)}(B') - \frac{\mathrm{slack}^{(s)}(B')}{m} = \sum_{a \in N(B')} \mathrm{capacity}(a)\left(Z_a - \frac{1}{m}\right).$$

The variables $\{Z_a\}_{a \in N(B')}$ are negatively associated, and each summand on the right-hand side has zero expectation and a range of length $\mathrm{capacity}(a)$. Hence, if $\mathrm{slack}^{(s)}(B') > 0$, Hoeffding's inequality gives

$$\begin{aligned}&\Pr\left[\mathrm{slack}^{(s,i)}(B') < 0\right] \\ &= \Pr\left[\sum_{a \in N(B')} \mathrm{capacity}(a)\left(Z_a - \frac{1}{m}\right) < -\frac{\mathrm{slack}^{(s)}(B')}{m}\right] \\ &\le \exp\left(-\frac{2\,\mathrm{slack}^{(s)}(B')^2}{m^2 \sum_{a \in N(B')} \mathrm{capacity}(a)^2}\right).\end{aligned}$$

□

A.2 Proof of Proposition 3.2

Proposition A.2 (Expected relative underdelivery). *Suppose that $G^{(s)}$ has an allocation $x = (x_{ab})_{(a,b) \in E^{(s)}}$ that exactly satisfies every demand. For this allocation, define $L \ge 1$ such that $1/L$ is the largest fraction of any demand contributed by a single supply:*

$$\frac{1}{L} = \max_{a \in A^{(s)},\, b \in B^{(s)}} \frac{M_{ab}x_{ab}}{\mathrm{demand}(b)}.$$

Under the randomized partition of the supply set $A^{(s)}$, define the underdelivery of demand b in subproblem $G^{(s,i)}$ as

$$u_b^{(s,i)} = \left[\frac{\mathrm{demand}(b)}{m} - \sum_{a \in A^{(s,i)}} M_{ab}x_{ab}\right]_+.$$

Then the expected relative underdelivery after partitioning satisfies

$$\frac{\mathbb{E}\left[\sum_{i=1}^m \sum_{b \in B^{(s)}} u_b^{(s,i)}\right]}{\sum_{b \in B^{(s)}} \mathrm{demand}(b)} \le \frac{1}{2}\sqrt{\frac{m-1}{L}}.$$

Proof. Fix $b \in B^{(s)}$ and $i \in \{1, \dots, m\}$. Since every supply is assigned to $A^{(s,i)}$ with probability $1/m$ and x exactly satisfies demand b,

$$\begin{aligned}\mathbb{E}\left[\sum_{a \in A^{(s,i)}} M_{ab}x_{ab}\right] &= \frac{1}{m}\sum_{a \in A^{(s)}} M_{ab}x_{ab} \\ &= \frac{\mathrm{demand}(b)}{m}.\end{aligned}$$

Because $A^{(s,i)}$ is a uniformly random subset of fixed size, the inclusion events of different supplies have nonpositive covariances. Therefore,

$$\mathrm{Var}\left[\sum_{a \in A^{(s,i)}} M_{ab}x_{ab}\right] \le \frac{m-1}{m^2}\sum_{a \in A^{(s)}} (M_{ab}x_{ab})^2.$$

Thus, the expected allocation to b in each subproblem equals its assigned demand. Its expected excess above $\mathrm{demand}(b)/m$ therefore equals its expected underdelivery below this value. Since their sum is the absolute deviation, the definition of $u_b^{(s,i)}$ gives

$$\begin{aligned}\mathbb{E}\left[u_b^{(s,i)}\right] &= \frac{1}{2}\mathbb{E}\left[\left|\sum_{a \in A^{(s,i)}} M_{ab}x_{ab} - \frac{\mathrm{demand}(b)}{m}\right|\right] \\ &\le \frac{1}{2}\sqrt{\mathrm{Var}\left[\sum_{a \in A^{(s,i)}} M_{ab}x_{ab}\right]} \\ &\le \frac{\sqrt{m-1}}{2m}\sqrt{\sum_{a \in A^{(s)}} (M_{ab}x_{ab})^2},\end{aligned}$$

where the first inequality follows from the Cauchy–Schwarz inequality. By the definition of L, each supply contributes at most $\mathrm{demand}(b)/L$ to demand b. Hence,

$$\begin{aligned}\sum_{a \in A^{(s)}} (M_{ab}x_{ab})^2 &\le \frac{\mathrm{demand}(b)}{L}\sum_{a \in A^{(s)}} M_{ab}x_{ab} \\ &= \frac{\mathrm{demand}(b)^2}{L}.\end{aligned}$$

Combining the preceding bounds gives

$$\mathbb{E}\left[u_b^{(s,i)}\right] \le \frac{\text{demand}(b)}{2m}\sqrt{\frac{m-1}{L}}.$$

Summing over all $i \in \{1,\dots,m\}$ and $b \in B^{(s)}$ yields

$$\mathbb{E}\left[\sum_{i=1}^{m}\sum_{b\in B^{(s)}} u_b^{(s,i)}\right] \le \frac{1}{2}\sqrt{\frac{m-1}{L}}\sum_{b\in B^{(s)}}\text{demand}(b).$$

Dividing both sides by the total demand proves the result. □

B Proof of Demand Adjustment Phase

Proposition B.1 (Approximation guarantee for demand adjustment). *Let Δ denote the total deficit reduction achieved by the demand adjustment phase. Let $\Delta^\star$ denote the maximum total deficit reduction achievable from the same initial allocation by applying admissible reallocation chains. Then*

$$\Delta \ge \frac{1}{3}\Delta^\star.$$

Proof. For $a \in A$ and $b \in B$, let r_a^0 and x_{ab}^0 denote the values of r_a and x_{ab} at the beginning of the demand adjustment phase, respectively, and let r_a and x_{ab} denote their values at the end of the phase. For $b \in D^{\text{unsat}}$, let q_b^0 and q_b denote the deficit of b at the beginning and end of the phase, respectively. Let $\mathcal{P}$ be the candidate set of all reallocation chains admissible. For each $P \in \mathcal{P}$, write

$$P = (a_0^P, b_0^P, a_1^P, b_1^P).$$

Suppose that the demand adjustment phase successively applies $P_1,\dots,P_T \in \mathcal{P}$. Its total reallocation amount is

$$\Delta = \sum_{t=1}^{T}\delta(P_t).$$

For each $P \in \mathcal{P}$, let $\Delta_P \ge 0$ denote the amount reallocated along P. The maximum feasible reallocation amount $\Delta^\star$ is defined by

$$\Delta^\star = \max_{\{\Delta_P\ge 0\}_{P\in\mathcal{P}}}\sum_{P\in\mathcal{P}}\Delta_P,$$

subject to

$$\sum_{\substack{P\in\mathcal{P}\\ a_0^P=a}}\Delta_P \le r_a^0, \qquad \forall a \in S^{\text{rem}},$$

$$\sum_{\substack{P\in\mathcal{P}\\ (a_1^P,b_0^P)=(a,b)}}\Delta_P \le x_{ab}^0, \qquad \forall a \in S^{\text{adj}},\ b \in D^{\text{sat}},$$

$$\sum_{\substack{P\in\mathcal{P}\\ b_1^P=b}}\Delta_P \le q_b^0, \qquad \forall b \in D^{\text{unsat}}.$$

Let $\{\Delta_P^\star\}_{P\in\mathcal{P}}$ be an optimal solution and define

$$\mathcal{P}^\star = \left\{P \in \mathcal{P} \;\middle|\; \Delta_P^\star > 0\right\}.$$

Thus, $\mathcal{P}^\star$ is the set of chains used by the optimal reallocation. Moreover,

$$\Delta^\star = \sum_{P\in\mathcal{P}^\star}\Delta_P^\star.$$

At the end of the demand adjustment phase, no admissible chain remains. Therefore, every $P \in \mathcal{P}^\star$ satisfies

$$r_{a_0^P} = 0 \quad \text{or} \quad x_{a_1^P b_0^P} = 0 \quad \text{or} \quad q_{b_1^P} = 0.$$

To obtain a disjoint partition of $\mathcal{P}^\star$, define

$$\mathcal{P}_r = \left\{P \in \mathcal{P}^\star \;\middle|\; r_{a_0^P} = 0\right\},$$

$$\mathcal{P}_x = \left\{P \in \mathcal{P}^\star \;\middle|\; r_{a_0^P} > 0,\ x_{a_1^P b_0^P} = 0\right\},$$

and

$$\mathcal{P}_q = \left\{P \in \mathcal{P}^\star \;\middle|\; r_{a_0^P} > 0,\ x_{a_1^P b_0^P} > 0,\ q_{b_1^P} = 0\right\}.$$

The termination condition ensures that these sets are pairwise disjoint and satisfy

$$\mathcal{P}^\star = \mathcal{P}_r \cup \mathcal{P}_x \cup \mathcal{P}_q.$$

Consequently,

$$\Delta^\star = \sum_{P\in\mathcal{P}_r}\Delta_P^\star + \sum_{P\in\mathcal{P}_x}\Delta_P^\star + \sum_{P\in\mathcal{P}_q}\Delta_P^\star.$$

It follows from the definition of $\Delta^\star$ that

$$\sum_{P\in\mathcal{P}_r}\Delta_P^\star \le \sum_{\substack{a\in S^{\text{rem}}\\ r_a=0}} r_a^0,$$

$$\sum_{P\in\mathcal{P}_x}\Delta_P^\star \le \sum_{\substack{a\in S^{\text{adj}},\, b\in D^{\text{sat}}\\ x_{ab}=0}} x_{ab}^0,$$

and

$$\sum_{P\in\mathcal{P}_q}\Delta_P^\star \le \sum_{\substack{b\in D^{\text{unsat}}\\ q_b=0}} q_b^0.$$

Each reallocation of amount $\delta(P_t)$ decreases one residual capacity in S^{rem}, one allocation from S^{adj} to D^{sat}, and one deficit in D^{unsat} by exactly $\delta(P_t)$. Summing over all applied chains gives

$$\sum_{a\in S^{\text{rem}}}(r_a^0 - r_a) = \Delta,$$

$$\sum_{a\in S^{\text{adj}},\, b\in D^{\text{sat}}}(x_{ab}^0 - x_{ab}) = \Delta,$$

$$\sum_{b\in D^{\text{unsat}}}(q_b^0 - q_b) = \Delta.$$

Restricting these sums to quantities exhausted at the end of the phase yields

$$\sum_{\substack{a\in S^{\text{rem}}\\ r_a=0}} r_a^0 \le \Delta,$$

$$\sum_{\substack{a\in S^{\text{adj}},\, b\in D^{\text{sat}}\\ x_{ab}=0}} x_{ab}^0 \le \Delta,$$

and

$$\sum_{\substack{b\in D^{\text{unsat}}\\ q_b=0}} q_b^0 \le \Delta.$$

Therefore, each of the three terms in the decomposition of $\Delta^\star$ is at most Δ, and hence

$$\Delta^\star = \sum_{P\in\mathcal{P}_r} \Delta_P^\star + \sum_{P\in\mathcal{P}_x} \Delta_P^\star + \sum_{P\in\mathcal{P}_q} \Delta_P^\star \le 3\Delta.$$

Equivalently,

$$\Delta \ge \frac{1}{3}\Delta^\star.$$

□

C Proof of heat replacement Phase

Proposition C.1 (local optimality of heat replacement). *At the end of the heat replacement phase, for every heat replacement P and every $0 < \eta \le \delta(P)$, reallocating η units along P cannot improve the offline objective.*

Proof. At termination, consider any heat replacement $P = (a_0, b, a_1)$. For $0 \le \eta \le \delta(P)$, let $g_P(\eta)$ denote the objective change obtained by reallocating η units along P. Then

$$g_P(\eta) = 2\eta\big(w_{a_1} r_{a_1} - w_{a_0} r_{a_0}\big) + \eta^2\big(w_{a_0} + w_{a_1}\big).$$

Since $w_{a_0} + w_{a_1} > 0$, $g_P(\eta)$ is convex on $[0, \delta(P)]$. Moreover,

$$g_P(0) = 0.$$

Since no admissible replacement remains at termination,

$$g_P(\delta(P)) = \Delta f(P) \le 0.$$

Therefore, for every $0 \le \eta \le \delta(P)$,

$$g_P(\eta) \le \frac{\eta}{\delta(P)} g_P(\delta(P)) \le 0.$$

Thus, no candidate P admits a feasible replacement amount that improves the offline objective after the heat replacement phase. □

D Proof of Cross-Subproblem Recovery

Proposition D.1 (total-allocation guarantee after cross–subproblem recovery). *Suppose that the direct allocation step and the subsequent global demand adjustment phase are both run to termination. Let Δ denote the total allocation in the final solution obtained on $G^{(s)}$ after cross-subproblem recovery, and let $\Delta^\star$ denote the maximum total allocation achievable by any feasible allocation on $G^{(s)}$ that contains no overdelivery. Then*

$$\Delta \ge \frac{2}{3}\Delta^\star.$$

Proof. Let

$$x^0 = \{x^0_{ab}\}_{(a,b)\in E^{(s)}} \qquad \text{and} \qquad x = \{x_{ab}\}_{(a,b)\in E^{(s)}}$$

denote the allocations immediately after the direct allocation step in cross-subproblem recovery and at the end of the subsequent global demand adjustment phase, respectively. For each $a \in A^{(s)}$ and $b \in B^{(s)}$, let r^0_a and q^0_b denote the residual capacity and demand deficit associated with x^0, and let r_a and q_b denote their values associated with x.

At the beginning of the global demand adjustment phase, define the following fixed sets:

$$\begin{aligned} D_{\mathrm{G}}^{\mathrm{unsat}} &= \{b \in B^{(s)} : q^0_b > 0\},\\ D_{\mathrm{G}}^{\mathrm{sat}} &= B^{(s)} \setminus D_{\mathrm{G}}^{\mathrm{unsat}},\\ S_{\mathrm{G}}^{\mathrm{rem}} &= \{a \in A^{(s)} : r^0_a > 0\},\\ S_{\mathrm{G}}^{\mathrm{adj}} &= \left\{a \in A^{(s)} : r^0_a = 0,\ \exists b \in D_{\mathrm{G}}^{\mathrm{sat}} \text{ such that } x^0_{ab} > 0\right\}. \end{aligned}$$

These sets remain fixed throughout the global demand adjustment phase. During this phase, a reallocation chain

$$P = (a_0, b_0, a_1, b_1) \in S_{\mathrm{G}}^{\mathrm{rem}} \times D_{\mathrm{G}}^{\mathrm{sat}} \times S_{\mathrm{G}}^{\mathrm{adj}} \times D_{\mathrm{G}}^{\mathrm{unsat}}$$

is admissible if

$$(a_0, b_0), (a_1, b_0), (a_1, b_1) \in E^{(s)}$$

and

$$r_{a_0} > 0, \qquad x_{a_1 b_0} > 0, \qquad q_{b_1} > 0.$$

The global demand adjustment phase terminates when no admissible reallocation chain remains. At the termination of the direct allocation step,

$$r^0_a = 0 \quad \text{or} \quad q^0_b = 0, \qquad \forall (a,b) \in E^{(s)}.$$

During global demand adjustment, residual capacities and demand deficits do not increase. Therefore,

$$r^0_a \ge r_a, \qquad \forall a \in A^{(s)},$$

and

$$q^0_b \ge q_b, \qquad \forall b \in B^{(s)}.$$

Moreover, only supplies in $S_{\mathrm{G}}^{\mathrm{rem}}$ can receive additional allocation to demands in $D_{\mathrm{G}}^{\mathrm{sat}}$. Hence,

$$x^0_{ab} \ge x_{ab}, \qquad a \notin S_{\mathrm{G}}^{\mathrm{rem}}, \quad b \in D_{\mathrm{G}}^{\mathrm{sat}}.$$

By the definition of Δ,

$$\Delta = \sum_{a\in A^{(s)}} \sum_{b\in B^{(s)}} M_{ab} x_{ab}.$$

Let

$$x^\star = \{x^\star_{ab}\}_{(a,b)\in E^{(s)}}$$

be a feasible allocation attaining the maximum total allocation on $G^{(s)}$. Then

$$\Delta^\star = \sum_{a\in A^{(s)}} \sum_{b\in B^{(s)}} M_{ab} x^\star_{ab}.$$

By construction, x contains no overdelivery, and $x^\star$ contains no overdelivery by the definition of $\Delta^\star$. We next characterize the additional allocation in $x^\star$ relative to x. The difference between $x^\star$ and x admits the standard alternating decomposition into simple paths that increase the total allocation and components that preserve the total allocation. The latter components can be discarded. Let $\mathcal{C}$ denote the remaining alternating reallocation paths. For each $C \in \mathcal{C}$, write

$$C = (a^C_0, b^C_0, a^C_1, b^C_1, \ldots, a^C_{\ell_C}, b^C_{\ell_C}),$$

where $\ell_C \ge 0$ is the number of allocation edges from which allocation is removed. Such a path satisfies

$$(a^C_i, b^C_i) \in E^{(s)}, \qquad i = 0, \ldots, \ell_C,$$
$$(a^C_i, b^C_{i-1}) \in E^{(s)}, \qquad x_{a^C_i b^C_{i-1}} > 0, \qquad i = 1, \ldots, \ell_C,$$

and

$$r_{a_0^C} > 0, \qquad q_{b_{\ell_C}^C} > 0.$$

For each $C \in \mathcal{C}$, let $\Delta_C > 0$ denote the amount associated with C in this decomposition. Then

$$\Delta^\star - \Delta = \sum_{C \in \mathcal{C}} \Delta_C.$$

Moreover, the total amount removed from any allocation edge cannot exceed its current allocation. Therefore,

$$\sum_{\substack{C \in \mathcal{C},\ 1 \le i \le \ell_C \\ (a_i^C, b_{i-1}^C) = (a,b)}} \Delta_C \le x_{ab}, \qquad \forall (a,b) \in E^{(s)}.$$

We next show that every path in C removes allocation from at least two edges. Suppose first that $\ell_C = 0$. Then

$$C = (a_0^C, b_0^C),$$

where

$$(a_0^C, b_0^C) \in E^{(s)}, \qquad r_{a_0^C} > 0, \qquad q_{b_0^C} > 0.$$

Since residual capacities and demand deficits do not increase,

$$r^0_{a_0^C} > 0, \qquad q^0_{b_0^C} > 0,$$

contradicting the termination condition of the direct allocation step. Therefore, $\ell_C \neq 0$. Suppose next that $\ell_C = 1$. Write

$$C = (a_0, b_0, a_1, b_1).$$

By the definition of an alternating reallocation path,

$$(a_0, b_0), (a_1, b_0), (a_1, b_1) \in E^{(s)}$$

and

$$r_{a_0} > 0, \qquad x_{a_1 b_0} > 0, \qquad q_{b_1} > 0.$$

Since residual capacities and demand deficits do not increase,

$$r^0_{a_0} > 0, \qquad q^0_{b_1} > 0.$$

Therefore,

$$a_0 \in S_{\mathrm{G}}^{\mathrm{rem}} \qquad \text{and} \qquad b_1 \in D_{\mathrm{G}}^{\mathrm{unsat}}.$$

Since $(a_0, b_0) \in E^{(s)}$ and $r^0_{a_0} > 0$, the termination condition of the direct allocation step gives

$$q^0_{b_0} = 0,$$

and hence

$$b_0 \in D_{\mathrm{G}}^{\mathrm{sat}}.$$

Similarly, since $(a_1, b_1) \in E^{(s)}$ and $q^0_{b_1} > 0$, we have

$$r^0_{a_1} = 0.$$

Thus, $a_1 \notin S_{\mathrm{G}}^{\mathrm{rem}}$. Since only supplies in $S_{\mathrm{G}}^{\mathrm{rem}}$ can receive additional allocation to demands in $D_{\mathrm{G}}^{\mathrm{sat}}$,

$$x^0_{a_1 b_0} \ge x_{a_1 b_0} > 0.$$

Together with $r^0_{a_1} = 0$ and $b_0 \in D_{\mathrm{G}}^{\mathrm{sat}}$, this implies

$$a_1 \in S_{\mathrm{G}}^{\mathrm{adj}}.$$

Consequently,

$$(a_0, b_0, a_1, b_1) \in S_{\mathrm{G}}^{\mathrm{rem}} \times D_{\mathrm{G}}^{\mathrm{sat}} \times S_{\mathrm{G}}^{\mathrm{adj}} \times D_{\mathrm{G}}^{\mathrm{unsat}}.$$

At the end of the global demand adjustment phase, it still satisfies

$$r_{a_0} > 0, \qquad x_{a_1 b_0} > 0, \qquad q_{b_1} > 0.$$

Thus,

$$P = (a_0, b_0, a_1, b_1)$$

remains an admissible reallocation chain, contradicting the termination condition of the global demand adjustment phase. Therefore, $\ell_C \neq 1$. Hence, every path in C satisfies $\ell_C \ge 2$. It follows that

$$\begin{aligned}
2(\Delta^\star - \Delta) &= 2 \sum_{C \in \mathcal{C}} \Delta_C \\
&\le \sum_{C \in \mathcal{C}} \ell_C \Delta_C \\
&= \sum_{(a,b) \in E^{(s)}} \sum_{\substack{C \in \mathcal{C},\ 1 \le i \le \ell_C \\ (a_i^C, b_{i-1}^C) = (a,b)}} \Delta_C \\
&\le \sum_{(a,b) \in E^{(s)}} x_{ab} \\
&= \sum_{a \in A^{(s)}} \sum_{b \in B^{(s)}} M_{ab} x_{ab} \\
&= \Delta.
\end{aligned}$$

Therefore,

$$2\Delta^\star \le 3\Delta,$$

or equivalently,

$$\Delta \ge \frac{2}{3}\Delta^\star.$$

□

E Proof of Incremental Allocation

Proposition E.1 (Incremental allocation guarantee relative to reoptimization). *Let $b_1^{\mathrm{on}}, \dots, b_t^{\mathrm{on}}$ denote the first t online contracts indexed by their arrival order, and define $B_t = B^{\mathrm{off}} \cup \{b_1^{\mathrm{on}}, \dots, b_t^{\mathrm{on}}\}$. Let A_t denote the total allocation maintained after processing these online contracts, and let $A_t^\star$ denote the maximum total allocation obtainable by re-solving the entire allocation problem for all contracts in B_t without overdelivering any offline contract. For each $\tau \le t$, let R_τ denote the maximum amount that can still be allocated to b_τ^{on} through the admissible chains remaining when its processing stops. Then*

$$A_t \ge \frac{2}{3}\left(A_t^\star - \sum_{\tau=1}^{t} R_\tau\right).$$

In particular, if every arrival is processed until no admissible chain remains, then

$$A_t \ge \frac{2}{3} A_t^\star.$$

Proof. For each $b \in B^{\mathrm{off}}$, let u_b^{off} denote its underdelivery in the offline allocation. For each $\tau \le t$, let V_τ^{on} denote the allocation received by b_τ^{on}. Since the offline allocation contains no overdelivery and online reallocations preserve the total allocation of every existing contract,

$$A_t = \sum_{b \in B^{\mathrm{off}}} \left(\mathrm{demand}(b) - u_b^{\mathrm{off}}\right) + \sum_{\tau=1}^{t} V_\tau^{\mathrm{on}}.$$

$A_t^\star$ is given by

$$\max \sum_{a\in A}\sum_{b\in B_t} M_{ab}x_{ab}$$

subject to

$$\sum_{b\in B_t} M_{ab}x_{ab} \le \text{capacity}(a), \qquad \forall a \in A,$$

$$\sum_{a\in A} M_{ab}x_{ab} + u_b = \text{demand}(b), \qquad \forall b \in B^{\text{off}},$$

Consider the alternating reallocation paths defined in the proof of Proposition 3.3. For each such path C, let ℓ_C denote the number of current allocation edges from which allocation is removed. At the end of the offline stage, no path with $\ell_C \le 1$ remains by the termination conditions established in Proposition 3.3.

When b_τ^{on} arrives, the completed direct allocation step exhausts the residual capacity of every supply adjacent to b_τ^{on}. Therefore, no path with $\ell_C = 0$ can end at b_τ^{on}. A path with $\ell_C = 1$ ending at b_τ^{on} has the form

$$C = (a_0, b_0, a_1, b_\tau^{\text{on}}),$$

where b_0 is an existing contract, $r_{a_0} > 0$, and $x_{a_1 b_0} > 0$. By the definitions of S^{adj} and S^{rem} for the current arrival, such a path is exactly an admissible online reallocation chain. Hence, when the processing of b_τ^{on} stops, the maximum total augmentation obtainable through all remaining length-one paths ending at b_τ^{on} is R_τ.

We next show that subsequent arrivals cannot create or increase the augmentation available through a path with $\ell_C \le 1$ ending at an earlier contract. Since residual capacities never increase, no new path with $\ell_C = 0$ can be created.

First, consider the direct allocation for a later contract b_σ^{on}, where $\sigma > \tau$. Such an allocation increases only an edge $(a, b_\sigma^{\text{on}})$ for which a has positive residual capacity immediately before the update. Suppose that this edge creates or increases a length-one path ending at an earlier contract b. Then a must also be eligible for b. Since residual capacities never increase, a already had positive residual capacity when the processing of b stopped. If b is an online contract, this contradicts the completion of its direct allocation step. If b is an offline contract, this contradicts the absence of a path with $\ell_C = 0$ at the end of the offline stage. Therefore, a subsequent direct allocation cannot create or increase a length-one path ending at an earlier contract.

Next, consider a subsequent online reallocation chain

$$P = (a_0, b_0, a_1, b_\sigma^{\text{on}}).$$

The only allocation edges increased by this chain are (a_0, b_0) and $(a_1, b_\sigma^{\text{on}})$. If (a_0, b_0) creates or increases a length-one path ending at an earlier contract b, then a_0 must be eligible for b. Since $r_{a_0} > 0$ before applying P, a path with $\ell_C = 0$ ending at b would already have existed when b was processed, which is a contradiction. The edge $(a_1, b_\sigma^{\text{on}})$ also cannot create or increase such a path, because this would require a residual supply adjacent to b_σ^{on}, whereas the completed direct allocation step for b_σ^{on} exhausted all such residual capacities. All other allocation edges are unchanged or decreased.

Consequently, every length-one path remaining after the first t arrivals and ending at b_τ^{on} was already available when the processing of b_τ^{on} stopped. Moreover, its residual-capacity and current-allocation resources cannot increase after that time. By the definition of R_τ, every feasible combination of such paths has total augmentation at most R_τ. The same argument shows that no length-one path ending at an offline contract can be created after the offline stage.

Now consider the alternating-path decomposition between the maintained allocation and an allocation attaining $A_t^\star$. Let $\mathcal{C}$ be the resulting set of paths, and let Δ_C denote the augmentation amount along each $C \in \mathcal{C}$. Then

$$A_t^\star - A_t = \sum_{C\in\mathcal{C}} \Delta_C.$$

Since no path with $\ell_C = 0$ remains and the paths with $\ell_C = 1$ are bounded by the corresponding remaining online reallocation chains,

$$\sum_{\substack{C\in\mathcal{C}\\ \ell_C=1}} \Delta_C \le \sum_{\tau=1}^{t} R_\tau.$$

Every other path satisfies $\ell_C \ge 2$. Since such a path removes Δ_C from ℓ_C current allocation edges, and the total amount removed from any edge cannot exceed its current allocation,

$$\sum_{\substack{C\in\mathcal{C}\\ \ell_C\ge 2}} \ell_C\Delta_C \le A_t.$$

Therefore,

$$\begin{aligned} 2(A_t^\star - A_t) &= 2\sum_{\substack{C\in\mathcal{C}\\ \ell_C=1}} \Delta_C + 2\sum_{\substack{C\in\mathcal{C}\\ \ell_C\ge 2}} \Delta_C \\ &\le 2\sum_{\tau=1}^{t} R_\tau + \sum_{\substack{C\in\mathcal{C}\\ \ell_C\ge 2}} \ell_C\Delta_C \\ &\le 2\sum_{\tau=1}^{t} R_\tau + A_t. \end{aligned}$$

Rearranging gives

$$A_t \ge \frac{2}{3}\left(A_t^\star - \sum_{\tau=1}^{t} R_\tau\right).$$

If every arrival is processed until no admissible online reallocation chain remains, then $R_\tau = 0$ for every $\tau \le t$. Hence,

$$A_t \ge \frac{2}{3}A_t^\star.$$

□

F Details of the Datasets

Table 9 reports additional dataset statistics, including the supply–demand ratio, sparsity, node degrees, and the distribution of heat values, which are defined as follows:

- **Supply–Demand Ratio**: defined as the number of supply types divided by the number of demand contracts.

- **Sparsity**: computed as $\frac{|E|}{|S|\cdot|D|}$, where S and D denote the sets of supply nodes and demand nodes, respectively, and $E \subseteq S \times D$ is the set of edges in the bipartite graph, with each edge indicating that a supply node is eligible for a demand node.
- **AvgDeg**: reports the average node degree, where the first value corresponds to supply nodes and the second to demand nodes.
- **Heat Distribution**: summarizes the normalized heat values on the supply side, partitioned into three bins: Low (heat ≤ 0.33), Medium ($0.33 <$ heat ≤ 0.67), and High ($0.67 <$ heat ≤ 1).
- **Mean / Std**: denote the mean and standard deviation of the heat values, respectively.

G Selling and Serving

Selling refers to selling GD impressions and allocating them to advertisers, which is the focus of this work.

Serving is the subsequent delivery stage. As impressions (users) arrive sequentially, each impression is assigned to an eligible contract according to the allocation computed during selling.

Although serving itself is beyond the scope of this work, we evaluate the impact of the GD allocations produced by our selling method on downstream serving performance under user traffic.

G.1 PILLS Serving Procedure

We adopt a simple serving procedure guided by the GD allocation computed during selling. When a user arrives and triggers an advertising request, the resulting impression is matched to a supply node a in the selling problem according to a predefined rule based on user features, such as age, region, device type, and demographic attributes.

At serving time t, let $d_b^{\text{rem}}(t)$ denote the remaining demand of contract b. For the arriving supply node a, define

$$B_a^{\text{rem}}(t) = \left\{ b \in B \;\middle|\; M_{ab} = 1,\ d_b^{\text{rem}}(t) > 0 \right\},$$

which denotes the set of contracts that are eligible for supply node a and still have remaining demand. We further define

$$x_a^{\text{total}} = \sum_{b \in B} M_{ab} x_{ab} \qquad \text{and} \qquad x_a^{\text{active}}(t) = \sum_{b \in B_a^{\text{rem}}(t)} x_{ab}.$$

Here, x_a^{total} denotes the total amount allocated from supply node a in the selling stage, whereas $x_a^{\text{active}}(t)$ denotes the total selling allocation from a to its eligible contracts that still have remaining demand at serving time t.

For each contract $b \in B$, the serving probability is defined as

$$\pi_{ab}(t) = \begin{cases} \dfrac{x_{ab}}{x_a^{\text{active}}(t)}, & \text{if } b \in B_a^{\text{rem}}(t) \text{ and } x_a^{\text{active}}(t) > 0, \\ \dfrac{1}{|B_a^{\text{rem}}(t)|}, & \text{if } b \in B_a^{\text{rem}}(t),\ x_a^{\text{total}} = 0, \text{ and } B_a^{\text{rem}}(t) \neq \emptyset, \\ 0, & \text{otherwise.} \end{cases}$$

The impression is assigned to a contract by sampling from this distribution. When $x_a^{\text{active}}(t) > 0$, the serving procedure follows the relative selling allocation of supply node a. Once a contract is fully delivered, it is removed from $B_a^{\text{rem}}(t)$, and the probabilities are renormalized over the remaining eligible contracts. If $x_a^{\text{total}} = 0$, meaning that a received no allocation during selling, the impression is assigned uniformly at random among the contracts in $B_a^{\text{rem}}(t)$. If $x_a^{\text{total}} > 0$ but $x_a^{\text{active}}(t) = 0$, or if $B_a^{\text{rem}}(t)$ is empty, the impression is not assigned to a GD contract.

G.2 Serving Experiment

G.2.1 ***Data.*** Serving performance is evaluated using user traffic on the same day in the selling procedure, which includes both normal-traffic periods and traffic-spike periods. As shown in Table 10, the *Selling Users* column reports the number of user impressions used in the selling procedure, whereas the *Serving Users* column reports the number of user impressions observed during the serving stage.

G.2.2 ***Metrics and Competitors.*** The goal of this experiment is to examine whether the quality of selling solutions translates into improved serving performance. We consider four selling configurations, namely Gurobi_WS–PILLS, HWM–FIFO, Rap–Rap, and PILLS–PILLS, each corresponding to a combination of offline and online algorithms used in the selling procedure.

We evaluate serving performance under two settings. For brevity, the selling component of each pipeline is denoted by its offline algorithm.

- **Original Serving.** Each selling solution uses its corresponding serving procedure, including **Gurobi-Selling/PILLS-Serving**, **Rap-Selling/Rap-Serving**, **HWM-Selling/HWM-Serving**, and **PILLS-Selling/PILLS-Serving**.
- **Unified Serving.** All selling solutions use the PILLS production serving procedure, including **Gurobi-Selling/PILLS-Serving**, **Rap-Selling/PILLS-Serving**, **HWM-Selling/PILLS-Serving**, and **PILLS-Selling/PILLS-Serving**.

We evaluate serving performance using $\#Ud$, the under-delivery rate defined in Section 4.2.1. Here, $\#Ud$ is computed from the serving outcomes.

G.2.3 ***Results.*** As shown in Table 10, PILLS/PILLS achieves the lowest under-delivery rate ($\#Ud$) across all evaluated datasets, demonstrating that the higher-quality selling-stage allocations produced by PILLS translate into better serving performance.

H The Analysis of Independent Subgraphs

Table 11 reports the statistics of independent subgraphs produced by the Level-1 division described in Section 3.1 across all datasets. For each dataset, we report the number of independent subgraphs, as well as the minimum and maximum subgraph sizes. The results show that all ten datasets can be decomposed into multiple independent subgraphs, with 5 to 14 subgraphs per dataset. The sizes of independent subgraphs vary substantially, ranging from 3452×3 to 39203258×422.

When the original graph consists of only one independent subgraph, or when the number of independent subgraphs exceeds the available number of threads, we directly apply Supply-side partitioning to the original graph, producing one subproblem per thread. Otherwise, each independent subgraph is further decomposed through Supply-side partitioning. For k independent subgraphs and T available threads, each subgraph is divided into $m = \max\{1, \lfloor T/k \rfloor\}$ subproblems, yielding a total of $km \leq T$ subproblems. This strategy exploits parallelism across and within independent subgraphs while keeping the number of subproblems within the available thread count.

Dataset	Supply	Demand	Supply–Demand Ratio	Sparsity	AvgDeg (S/D)	Heat Distribution				
						Low (%)	Medium (%)	High (%)	Mean	Std
$\mathcal{D}_1$	32505062	330	98500.19	0.0265	8.73 / 859906.60	34.2%	48.7%	17.1%	0.372	0.158
$\mathcal{D}_2$	87263918	1021	85469.07	0.0219	22.41 / 1915362.14	38.1%	45.6%	16.3%	0.361	0.149
$\mathcal{D}_3$	99824983	1023	97580.63	0.0253	25.89 / 2526362.08	31.7%	49.8%	18.5%	0.384	0.166
$\mathcal{D}_4$	73662802	1019	72289.31	0.0182	18.56 / 1341689.66	36.5%	46.3%	17.2%	0.368	0.154
$\mathcal{D}_5$	74228133	1020	72772.68	0.0204	20.83 / 1515854.54	39.4%	44.8%	15.8%	0.352	0.147
$\mathcal{D}_6$	78860455	1021	77238.45	0.0159	16.27 / 1256669.82	33.1%	48.2%	18.7%	0.377	0.161
$\mathcal{D}_7$	79975731	1018	78561.62	0.0191	19.45 / 1528023.63	37.6%	45.1%	17.3%	0.365	0.153
$\mathcal{D}_8$	77864997	1019	76413.15	0.0146	14.92 / 1140084.35	32.4%	47.7%	19.9%	0.389	0.168
$\mathcal{D}_9$	81020881	1023	79199.30	0.0173	17.68 / 1400243.96	38.6%	44.2%	17.2%	0.358	0.151
$\mathcal{D}_{10}$	86934221	1022	85062.84	0.0209	21.34 / 1815240.84	34.7%	46.9%	18.4%	0.375	0.157

Table 9: Statistical properties of the datasets.

Table 10: Serving underdelivery across different selling–serving pipelines. #Selling denotes the number of selling users, and #Serving denotes the number of serving users.

	#Selling	#Serving	Original Serving				Unified Serving			
			Gurobi_WS/PILLS	HWM/HWM	Rap/Rap	PILLS/PILLS	Gurobi_WS/PILLS	HWM/PILLS	Rap/PILLS	PILLS/PILLS
$\mathcal{D}_1$	32505062	31204860	0.036	0.000	0.259	**0.000**	0.036	0.038	0.260	**0.000**
$\mathcal{D}_2$	87263918	97735588	0.040	0.041	0.322	**0.029**	0.040	0.041	0.323	**0.029**
$\mathcal{D}_3$	99824983	117793480	0.038	0.038	0.320	**0.024**	0.038	0.039	0.325	**0.024**
$\mathcal{D}_4$	73662802	67769778	0.039	0.036	0.236	**0.025**	0.039	0.040	0.300	**0.025**
$\mathcal{D}_5$	74228133	77939540	0.041	0.041	0.319	**0.030**	0.041	0.042	0.318	**0.030**
$\mathcal{D}_6$	78860455	98575569	0.038	0.040	0.321	**0.028**	0.038	0.039	0.319	**0.028**
$\mathcal{D}_7$	79975731	77576459	0.040	0.037	0.320	**0.026**	0.040	0.041	0.321	**0.026**
$\mathcal{D}_8$	77864997	94995296	0.040	0.041	0.327	**0.030**	0.040	0.041	0.325	**0.030**
$\mathcal{D}_9$	81020881	72918793	0.039	0.040	0.322	**0.028**	0.039	0.040	0.320	**0.028**
$\mathcal{D}_{10}$	86934221	82514653	0.038	0.041	0.320	**0.029**	0.038	0.039	0.319	**0.029**
Average	–	–	0.039	0.036	0.307	**0.025**	0.039	0.040	0.313	**0.025**

Dataset	Independent Subgraphs	Min Size ($\lvert S\rvert \times \lvert D\rvert$)	Max Size ($\lvert S\rvert \times \lvert D\rvert$)
$\mathcal{D}_1$	5	3452×3	16949257×74
$\mathcal{D}_2$	14	4918×5	34258864×418
$\mathcal{D}_3$	11	5333×5	39203258×422
$\mathcal{D}_4$	9	4182×5	28799175×414
$\mathcal{D}_5$	13	4174×4	29180603×414
$\mathcal{D}_6$	12	4283×5	30974578×410
$\mathcal{D}_7$	8	4571×5	31304732×408
$\mathcal{D}_8$	10	4327×4	30571009×408
$\mathcal{D}_9$	14	4470×5	31826946×412
$\mathcal{D}_{10}$	13	4857×6	34068659×413

Table 11: The number of independent subgraphs and the minimum and maximum subgraph sizes, where $|S|$ and $|D|$ represent the sizes of the supply-node set and the demand-node set within an independent subgraph, respectively.

I Detailed Evaluation on Public Datasets

I.1 Datasets and Experimental Setup

We evaluate PILLS on three publicly available advertising datasets: AUAF [6], Criteo DAC [19], and Criteo Attribution [21]. The resulting GD allocation instances contain 20,735–115,050 supply nodes and 50–96 demand nodes, substantially smaller than our industrial datasets.

We compare PILLS with HWM, Gurobi, and Rap using the same offline–online evaluation metrics as in the main experiments.

I.2 Results

Table 12 reports the offline and online allocation results on the three public datasets. PILLS achieves the highest offline objective on all three datasets and the best or tied-best online performance.

Table 12: Offline and online allocation performance on public datasets.

Dataset	*HWM-FIFO*			*Gurobi_WS-FIFO*			*Rap-RAP*			*PILLS-PILLS*		
	Off.#Obj	***Off.#Ud***	***On.#Obj***	***Off.#Obj***	***Off.#Ud***	***On.#Obj***	***Off.#Obj***	***Off.#Ud***	***On.#Obj***	***Off.#Obj***	***Off.#Ud***	***On.#Obj***
AUAF	-384.95M	17.19%	20.483K	-342.67M	13.17%	22.147K	-371.62M	15.86%	27.315K	**-313.10**	**11.18%**	**30.762**
Criteo DAC	0.392	0	9.918	4.376	0	9.925	1.214	0	**9.995**	**4.582**	0	**9.995**
Criteo Attribution	1.770	0.0002%	45.061	14.342	0.0003%	41.471	4.863	0	46.735	**16.905**	0	**49.210**

Compared with the billion-scale industrial instances, competing methods achieve more competitive results on these smaller problems, illustrating the impact of problem scale on their performance.

Detailed experimental results and logs are available in our repository.[5]

[5]https://anonymous.4open.science/r/PILLS-C1F0